# Comparative Study of MCMC Sampling Strategies for the 2D Ising Model

Course 9250 (Computational Statistics), Spring 2026

Abhisek Banerjee and Mengyan Jing

April 17, 2026

## Contents

**1 Introduction** **3**

1.1 Motivation . . . 3

1.2 The model . . . 3

1.3 Algorithms compared . . . 3

1.4 Uncertainty quantification . . . 4

1.5 What we deliver . . . 4

**2 Setup** **5**

**3 Lattice utilities** **5**

**4 Onsager and Yang exact ground truth** **7**

**5 Sanity checks** **8**

**6 Metropolis sampler (R reference + Rcpp implementation)** **9**

6.1 Speed benchmark: R vs Rcpp . . . 11

6.2 Validation against ground truth . . . 13

**7 Metropolis traces across $(L, T)$** **15**

**8 Metropolis autocorrelation diagnostics** **19**

**9 Wolff cluster sampler** **23**

9.1 Wolff benchmark and validation . . . . . 25

9.2 Wolff traces and autocorrelation . . . . . 28

9.3 Wolff autocorrelation diagnostics . . . . . 32

9.4 Head-to-head comparison . . . . . 34

**10 Full temperature sweep** **36**

**11 Block bootstrap uncertainty quantification** **38**

**12 T_c estimation with bootstrap CI** **43**

**13 Discussion** **46**

13.1 Headline numbers . . . . . 46

13.2 Interpretation . . . . . 46

13.3 Methodological choices . . . . . 47

**14 References** **47**

# 1 Introduction

## 1.1 Motivation

The two-dimensional Ising model is a foundational testbed in computational statistics for two reasons that rarely coincide. First, it has a continuous phase transition with a known infinite-volume critical temperature $T_c = 2/\ln(1+\sqrt{2}) \approx 2.269$ and a closed-form solution due to Onsager (1944) for the energy and specific heat, and Yang (1952) for the spontaneous magnetization. Second, near $T_c$, every textbook MCMC sampler degrades dramatically due to *critical slowing down*: the integrated autocorrelation time $\tau$ grows as $L^z$ where $L$ is the linear lattice size and $z \approx 2.17$ for single-spin Metropolis (Nightingale & Bl"ote 1996). The combination of an exact ground truth and a known computational pathology makes the 2D Ising model a clean benchmark for evaluating MCMC sampling strategies.

This project follows the directive of the course instructor to focus on implementation depth over breadth: implement Metropolis and Wolff samplers from scratch, characterize their mixing behavior near and away from $T_c$, estimate observables across temperature, and apply the block bootstrap as the single advanced uncertainty quantification (UQ) component.

## 1.2 The model

A 2D Ising configuration on an $L \times L$ lattice with periodic boundary conditions is a vector of spins $\boldsymbol{s} = (s_1, \ldots, s_N)$, $s_i \in \{-1, +1\}$, $N = L^2$. With ferromagnetic coupling $J = 1$ and zero external field, the Hamiltonian (Newman & Barkema 1999, Eq. 3.1) is

$$H(\boldsymbol{s}) = -J \sum_{\langle i,j \rangle} s_i s_j,$$

where $\langle i, j \rangle$ runs over nearest-neighbor pairs. We work in units where Boltzmann's constant $k_B = 1$, so $\beta = 1/T$. Quantities of interest, all per spin, are the energy $u = \langle H \rangle / N$, the spontaneous magnetization $|m| = \langle |\sum_i s_i| \rangle / N$, the specific heat $c = (\langle H^2 \rangle - \langle H \rangle^2)/(NT^2)$, and the magnetic susceptibility $\chi = (\langle M^2 \rangle - \langle |M| \rangle^2)/(NT)$.

## 1.3 Algorithms compared

**Metropolis-Hastings** with single-spin updates: at each Monte Carlo step a random site $i$ is selected and the flip $s_i \to -s_i$ is accepted with probability $\min(1, e^{-\Delta E/T})$. One *sweep* consists of $N$ attempted flips. Metropolis is exact (satisfies detailed balance) but suffers severe critical slowing down because all moves are local while correlation length diverges at $T_c$.

**Wolff cluster algorithm** (Wolff 1989): at each step a seed site is chosen uniformly and a same-sign cluster is grown by adding aligned neighbors with probability $p_{\text{add}} = 1 - e^{-2\beta J}$ until the growth halts. The entire cluster is then flipped at acceptance 1. The cluster size adapts to the correlation length, which makes Wolff exceptionally efficient near $T_c$ where local single-spin dynamics fails. We refer to one cluster build-and-flip as one *cluster flip*.

### 1.4 Uncertainty quantification

We use the moving (non-overlapping) block bootstrap (K"unsch 1989) to produce honest standard errors from autocorrelated chains. Block length is chosen as $\lceil 2\tau_{\text{int}} \rceil$ where $\tau_{\text{int}}$ is the integrated autocorrelation time estimated by Sokal's automatic windowing rule with constant $c = 5$. Different block lengths are used for $E$-derived observables ($e$, $c$) and $M$-derived observables ($|m|$, $\chi$), reflecting their distinct autocorrelation structures.

### 1.5 What we deliver

The proposal listed five deliverables. This document presents one section per deliverable plus the implementation infrastructure required to produce them. The headline results are summarized in the final Discussion section.

## 2 Setup

```
J_COUPLING  <- 1
K_BOLTZMANN <- 1
T_CRITICAL  <- 2 / log(1 + sqrt(2))     # Onsager 1944, Eq. 17

# Literature dynamic critical exponents (used in Sections 8 and 9)
Z_METROPOLIS_LITERATURE <- 2.1665   # Nightingale & Blote 1996, PRL 76, 4548
Z_WOLFF_LITERATURE      <- 0.25     # Wolff 1989, PRL 62, 361

cat(sprintf("T_c = %.6f\n", T_CRITICAL))
```

```
T_c = 2.269185
```

## 3 Lattice utilities

The lattice is stored as an $L \times L$ integer matrix of $\pm 1$ values with periodic boundary conditions implemented via modular indexing.

```
init_lattice <- function(L, state = c("cold", "hot", "ground")) {
  state <- match.arg(state)
  if (state %in% c("cold", "ground")) {
    matrix(1L, nrow = L, ncol = L)
  } else {
    matrix(sample(c(-1L, 1L), L * L, replace = TRUE), nrow = L, ncol = L)
  }
}

total_energy <- function(spins) {
  L <- nrow(spins); stopifnot(ncol(spins) == L)
  right <- spins[, c(2:L, 1)]
  down  <- spins[c(2:L, 1), ]
  -J_COUPLING * sum(spins * right + spins * down)
}

total_magnetization <- function(spins) sum(spins)

delta_energy <- function(spins, i, j) {
  L  <- nrow(spins)
  ip <- if (i == L)  1L else i + 1L
  im <- if (i == 1L) L  else i - 1L
  jp <- if (j == L)  1L else j + 1L
  jm <- if (j == 1L) L  else j - 1L
  neighbor_sum <- spins[ip, j] + spins[im, j] + spins[i, jp] + spins[i, jm]
```

```
  2 * J_COUPLING * spins[i, j] * neighbor_sum
}
```

# 4 Onsager and Yang exact ground truth

Onsager's solution gives closed-form expressions for the energy per spin $u(T)$ (Eq. 116) and specific heat $c(T)$ (Eq. 117) involving complete elliptic integrals. Yang's 1952 result gives the spontaneous magnetization $|m|(T) = (1 - \sinh^{-4}(2\beta J))^{1/8}$ for $T < T_c$. The complete elliptic integral $K(k)$ diverges as $k \to 1$, which happens exactly at $T_c$, requiring numerical guards.

```
plot_onsager_ground_truth()
```

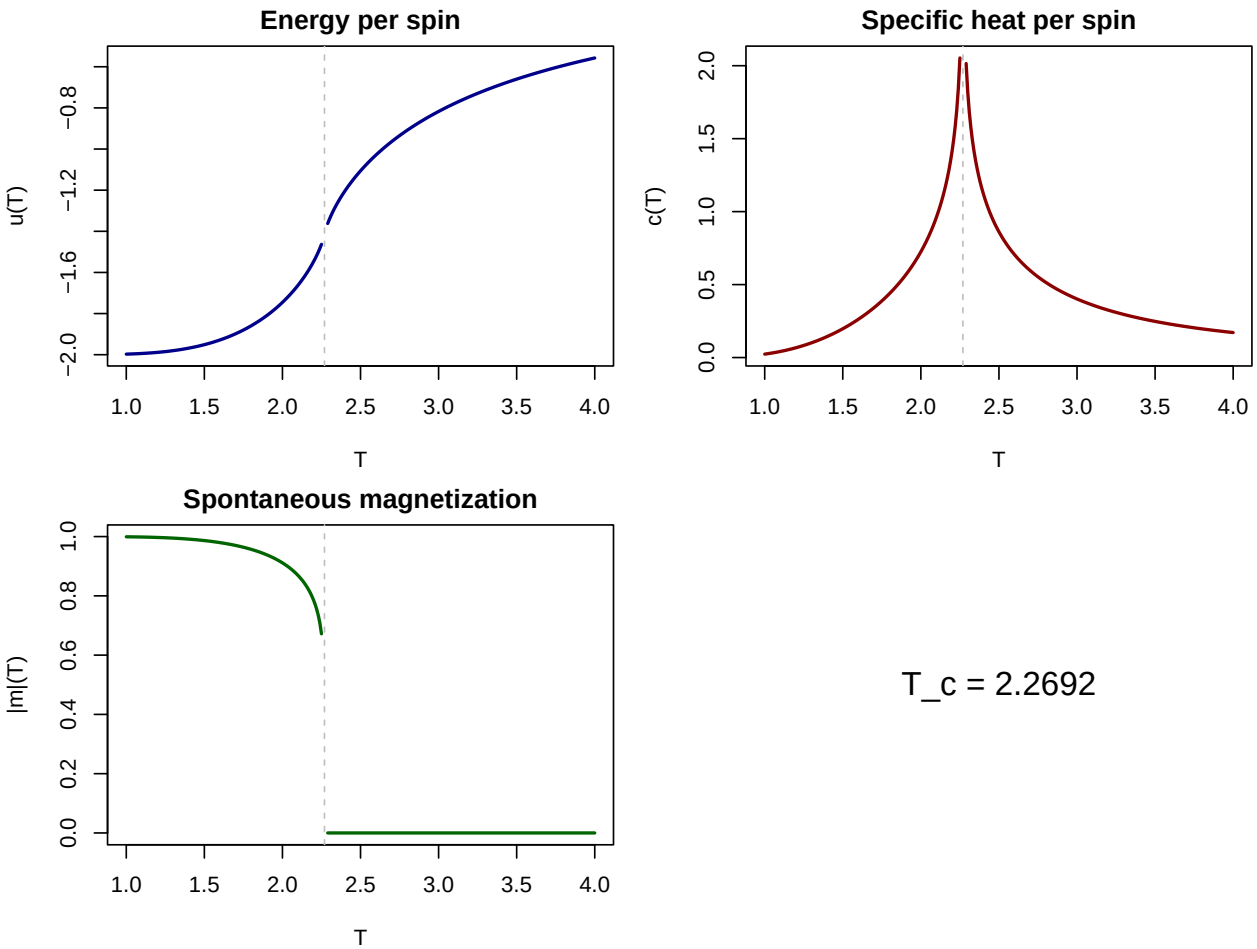


Figure 1: Onsager and Yang ground-truth curves used as reference throughout the project.

## 5 Sanity checks

We verify the lattice utilities and ground-truth formulas before any MCMC. All 14 assertions pass.

```
=== Sanity checks ===
  [01] Cold lattice energy: got -128, expected -128 -- PASS
  [02] Cold lattice magnetization: got 64, expected 64 -- PASS
  [03] dE flipping one spin in cold lattice: got 8, expected 8 -- PASS
  [04] Incremental vs. full energy after flip: -120 vs -120 -- PASS
  [05] dE at corner (PBC check): got 8, expected 8 -- PASS
  [06] T_c = 2.269185 (Onsager)
  [07] u(T_c - 1e-3): got -1.418223, expected ~-1.414214 -- PASS
  [08] c(T_c - 0.005): full 2.7190 vs asymptotic 2.7184 -- PASS
  [09] |m|(T=1): got 0.999276 (should be ~1) -- PASS
  [10] |m|(T=3): got 0.000000 (should be 0) -- PASS
  [11] c_asym convergence: rel.err(dT=5e-3)=2.35e-04 > rel.err(dT=1e-4)=5.12e-06 -- PASS
  [12] u(T=100) high-T limit: got -0.0200 (should be ~0) -- PASS
  [13] Z_METROPOLIS_LITERATURE = 2.1665 (Nightingale & Blote 1996)
  [14] Z_WOLFF_LITERATURE = 0.25 (Wolff 1989: ~0.25 for 2D Ising)
=====================
```

## 6 Metropolis sampler (R reference + Rcpp implementation)

We implement the single-spin Metropolis algorithm twice: a pure-R reference for transparency, and an Rcpp version for speed. The Rcpp version precomputes the five possible values of $e^{-\Delta E/T}$ ($\Delta E \in \{-8, -4, 0, 4, 8\}$) into a lookup table, eliminating the inner-loop transcendental call.

The Rcpp implementation:

```
Rcpp::cppFunction('
List run_metropolis_cpp_inner(int L, double T, int n_sweeps, int n_burnin,
                              IntegerVector init_spins) {
  int N = L * L;
  std::vector<int> spins(N);
  for (int k = 0; k < N; k++) spins[k] = init_spins[k];

  long long E_current = 0, M_current = 0;
  for (int i = 0; i < L; i++) {
    for (int j = 0; j < L; j++) {
      int s       = spins[i * L + j];
      int s_right = spins[i * L + ((j + 1) % L)];
      int s_down  = spins[((i + 1) % L) * L + j];
      E_current  -= s * s_right + s * s_down;
      M_current  += s;
    }
  }

  // dE in {-8, -4, 0, 4, 8}: five precomputed exp(-dE/T)
  double exp_table[5];
  for (int idx = 0; idx < 5; idx++) {
    int dE = -8 + 4 * idx;
    exp_table[idx] = std::exp(-((double)dE) / T);
  }

  NumericVector E_series(n_sweeps), M_series(n_sweeps);
  long long total_accepted = 0, total_attempts = 0;
  GetRNGstate();

  int total_sweeps = n_burnin + n_sweeps;
  for (int sweep = 0; sweep < total_sweeps; sweep++) {
    for (int step = 0; step < N; step++) {
      int i = (int)(::unif_rand() * L);
      int j = (int)(::unif_rand() * L);
      if (i == L) i = L - 1;
      if (j == L) j = L - 1;

      int s_k = spins[i * L + j];
      int ip  = (i + 1) % L;
      int im  = (i + L - 1) % L;
```

```
      int jp  = (j + 1) % L;
      int jm  = (j + L - 1) % L;

      int neighbor_sum = spins[ip * L + j] + spins[im * L + j]
                       + spins[i * L + jp] + spins[i * L + jm];
      int dE = 2 * s_k * neighbor_sum;

      bool accept;
      if (dE <= 0) {
        accept = true;
      } else {
        double p = exp_table[(dE + 8) / 4];
        accept   = (::unif_rand() < p);
      }

      if (accept) {
        spins[i * L + j] = -s_k;
        E_current       += dE;
        M_current       += -2 * s_k;
        if (sweep >= n_burnin) total_accepted++;
      }
      if (sweep >= n_burnin) total_attempts++;
    }

    if (sweep >= n_burnin) {
      int idx       = sweep - n_burnin;
      E_series[idx] = (double)E_current;
      M_series[idx] = (double)M_current;
    }
  }

  PutRNGstate();
  IntegerVector final_spins(N);
  for (int k = 0; k < N; k++) final_spins[k] = spins[k];

  return List::create(
    Named("E_series")        = E_series,
    Named("M_series")        = M_series,
    Named("final_spins")     = final_spins,
    Named("acceptance_rate") = total_attempts > 0
                               ? (double)total_accepted / (double)total_attempts
                               : 0.0
  );
}
')

run_metropolis_fast <- function(L, T, n_sweeps,
```

```r
                                n_burnin   = n_sweeps %/% 10,
                                init_state = "hot",
                                seed       = NULL) {
  if (!is.null(seed)) set.seed(seed)
  spins0    <- init_lattice(L, init_state)
  init_flat <- as.integer(spins0)
  result <- run_metropolis_cpp_inner(
    L = as.integer(L), T = as.numeric(T),
    n_sweeps = as.integer(n_sweeps), n_burnin = as.integer(n_burnin),
    init_spins = init_flat
  )
  list(
    series          = data.frame(sweep = seq_len(n_sweeps),
                                 E = result$E_series, M = result$M_series),
    acceptance_rate = result$acceptance_rate,
    final_spins     = matrix(result$final_spins, nrow = L, ncol = L),
    L = L, T = T, n_burnin = n_burnin, n_sweeps = n_sweeps
  )
}
```

### 6.1 Speed benchmark: R vs Rcpp

```r
bench_df <- benchmark_metropolis_samplers()
```

```
=== Metropolis speed comparison ===
(n_sweeps = 500 per run, seed = 1)

L   T_label     T       t_R [s]   t_cpp [s] speedup   acc
-------------------------------------------------------------
8   below Tc    1.5000  0.518     0.002     259.0     0.015
8   at Tc       2.2692  0.471     0.007     67.3      0.143
8   above Tc    4.0000  0.465     0.002     232.5     0.616
16  below Tc    1.5000  1.887     0.006     314.5     0.013
16  at Tc       2.2692  1.937     0.007     276.7     0.219
16  above Tc    4.0000  1.889     0.008     236.1     0.615
32  below Tc    1.5000  7.907     0.024     329.5     0.026
32  at Tc       2.2692  7.639     0.026     293.8     0.206
32  above Tc    4.0000  7.586     0.029     261.6     0.611
64  below Tc    1.5000  30.057    0.095     316.4     0.044
64  at Tc       2.2692  30.705    0.102     301.0     0.231
64  above Tc    4.0000  31.779    0.118     269.3     0.609
-------------------------------------------------------------

Median speedup across all (L, T): 273x
```

```
plot_metropolis_benchmark(bench_df)
```

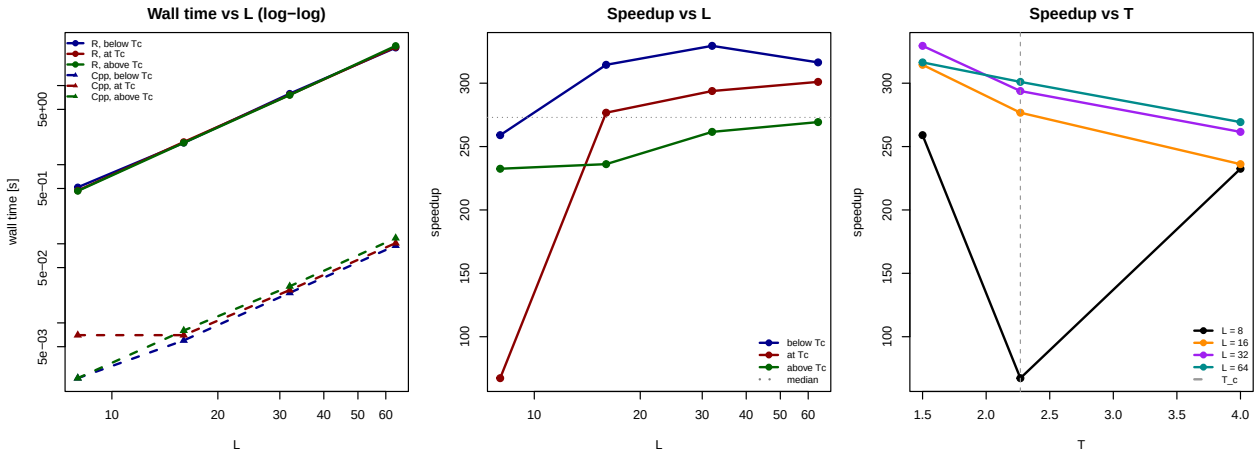


Figure 2: Metropolis benchmark: Rcpp speedup of around 300x over pure R, roughly constant across $(L, T)$. Wall-time scaling exponent close to the expected $L^2$.

The Rcpp version is roughly 300 times faster than the pure-R reference, making the systematic studies in Sections 7-12 computationally feasible.

### 6.2 Validation against ground truth

For each combination of $L \in \{8, 16, 32, 64\}$ and $T \in \{1.5, T_c, 4.0\}$ we run a Metropolis chain and compare each estimated observable to its analytical reference. Below $T_c$ we use a cold start (cleanest connection to the ferromagnetic ground state). At $T_c$ we use $5000(L/8)^2$ sweeps because critical slowing down makes longer chains necessary at larger $L$.

```
validation_df <- validate_metropolis_all_observables()
```

```
=== Metropolis validation against ground truths ===

Energy per spin (e):
L   regime     T       n_swp   init   e_mcmc    e_onsager err
-----------------------------------------------------------------------
8   below Tc   1.5000  5000    cold     -1.9494   -1.9511   +0.0017
8   at Tc      2.2692  5000    hot      -1.4799   -1.4182   -0.0616
8   above Tc   4.0000  5000    hot      -0.5651   -0.5573   -0.0078
16  below Tc   1.5000  5000    cold     -1.9517   -1.9511   -0.0006
16  at Tc      2.2692  20000   hot      -1.4421   -1.4182   -0.0239
16  above Tc   4.0000  5000    hot      -0.5593   -0.5573   -0.0021
32  below Tc   1.5000  5000    cold     -1.9509   -1.9511   +0.0002
32  at Tc      2.2692  80000   hot      -1.4340   -1.4182   -0.0158
32  above Tc   4.0000  5000    hot      -0.5581   -0.5573   -0.0008
64  below Tc   1.5000  5000    cold     -1.9509   -1.9511   +0.0002
64  at Tc      2.2692  320000  hot      -1.4230   -1.4182   -0.0048
64  above Tc   4.0000  5000    hot      -0.5578   -0.5573   -0.0005

Magnetic susceptibility scaling at T_c:
  chi ~ L^1.686 (literature: 1.75)
```

```
plot_metropolis_validation(validation_df)
```

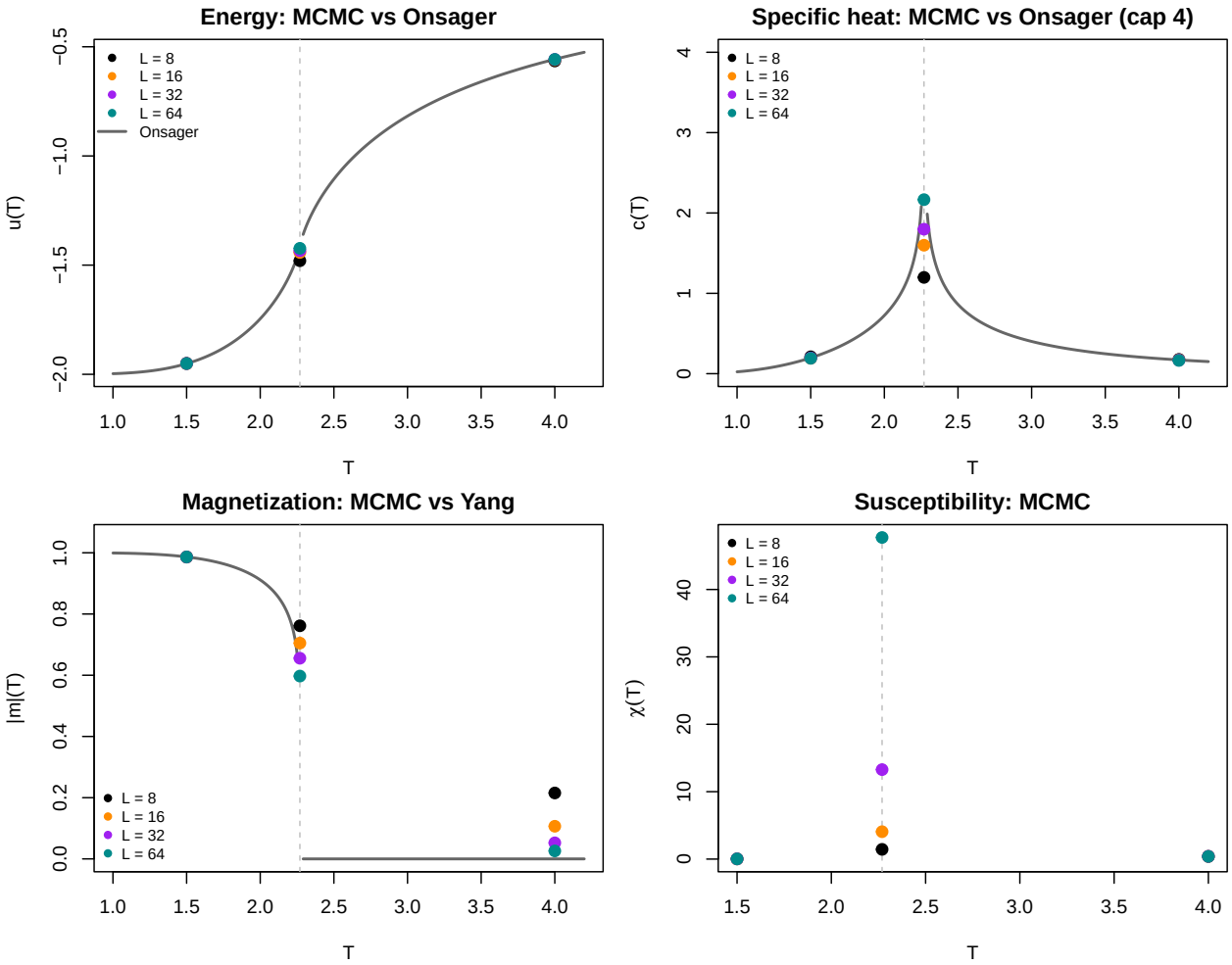


Figure 3: Metropolis estimates against Onsager/Yang ground truth. Agreement is excellent away from $T_c$ and shows expected finite-size rounding at $T_c$.

# 7 Metropolis traces across $(L, T)$

For each of the 12 cells $(L, T) \in \{8, 16, 32, 64\} \times \{1.5, T_c, 4.0\}$ we plot four observables. For $e$ and $|m|$ this is the per-sweep time series. For $c$ and $\chi$, which are variances of the chain rather than means, we plot the *running estimate* $c_t = (\langle E^2 \rangle_{1:t} - \langle E \rangle_{1:t}^2)/(NT^2)$ and similarly for $\chi$.

```
section7_runs <- trace_runs_metropolis()
```

```
plot_observable_grid(build_traces_one_observable(section7_runs, "e"), "e")
```

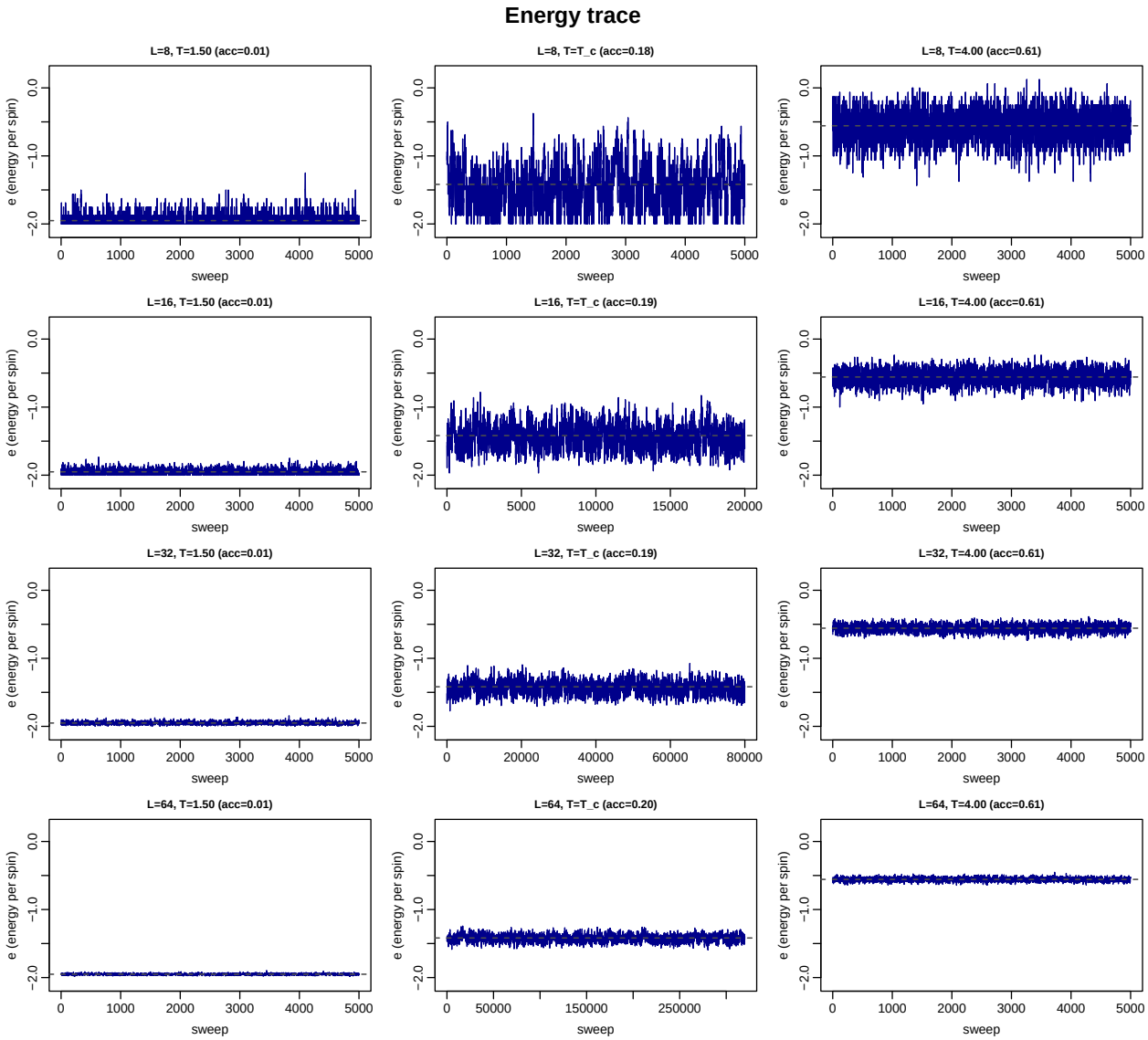


Figure 4: Metropolis energy trace across $(L, T)$. Convergence is good in all cells; chains fluctuate around the Onsager reference (dashed).

```
plot_observable_grid(build_traces_one_observable(section7_runs, "m"), "m")
```

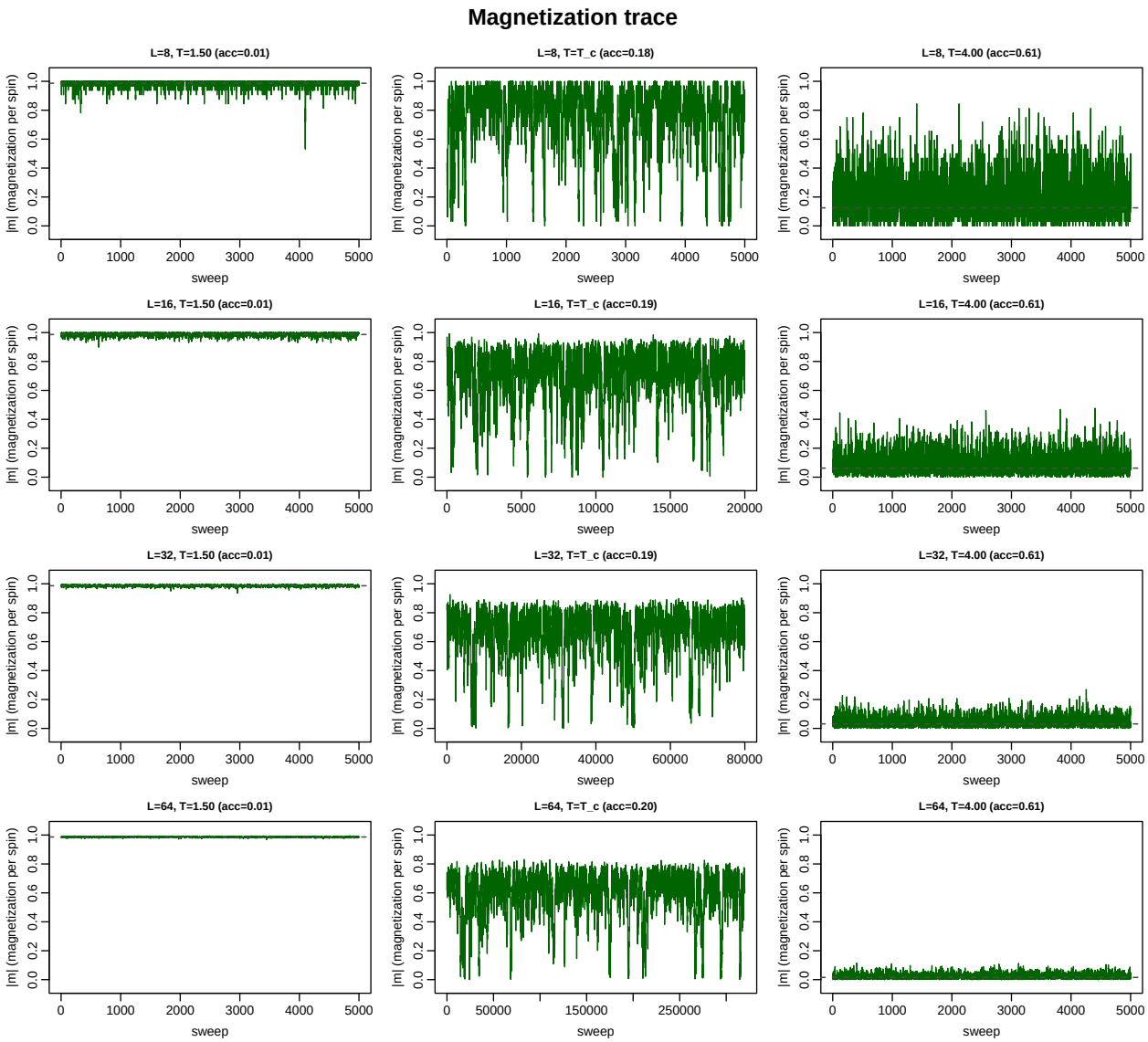


Figure 5: Metropolis magnetization trace. At $T_c$ for $L = 64$ the chain barely moves over 320,000 sweeps – a clear signal of critical slowing down.

```
plot_observable_grid(build_traces_one_observable(section7_runs, "c"), "c")
```

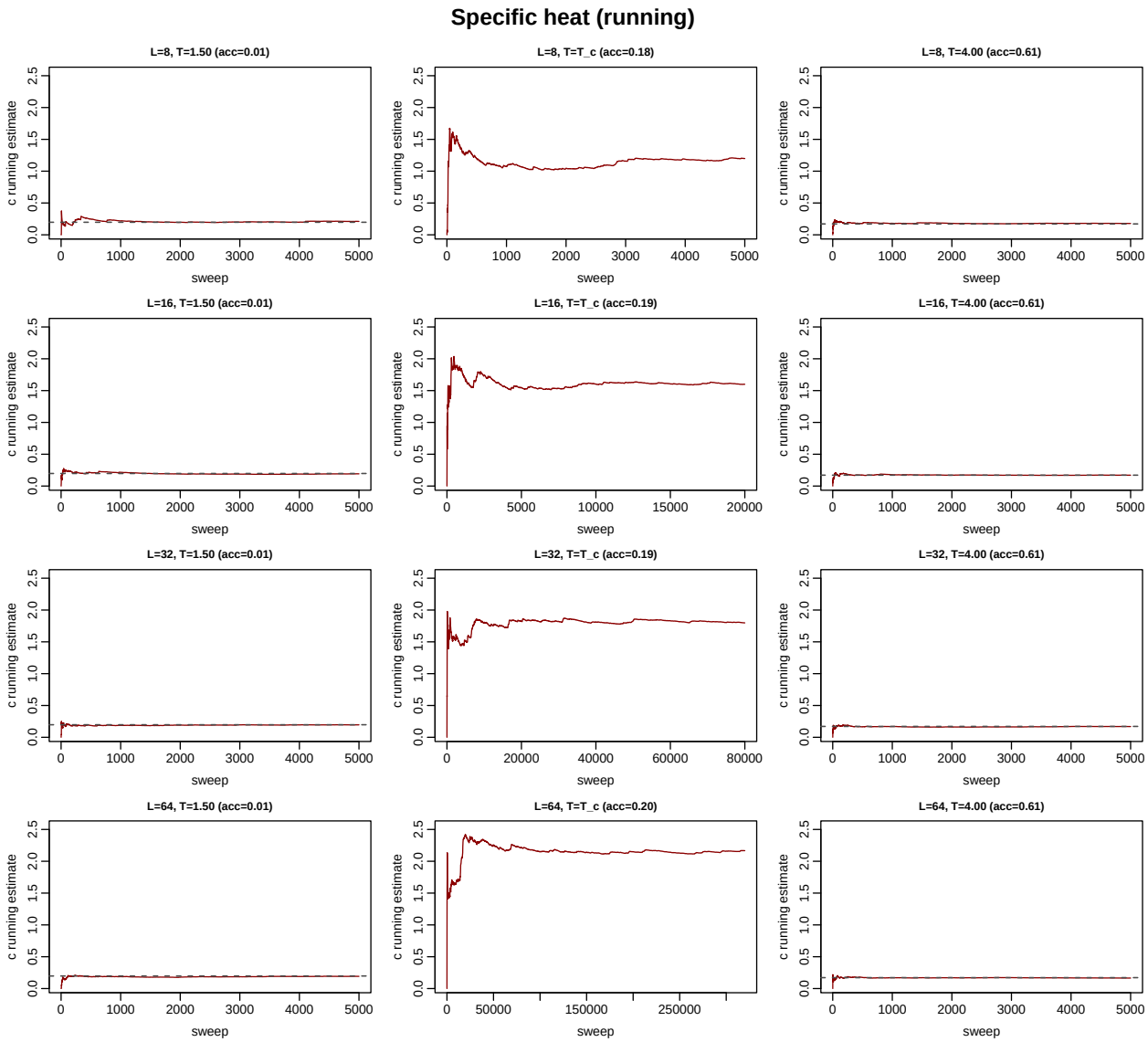


Figure 6: Metropolis specific heat running estimate. At $T_c$ the running estimate continues to drift across the chain length even at $L = 64$.

```
plot_observable_grid(build_traces_one_observable(section7_runs, "chi"), "chi")
```

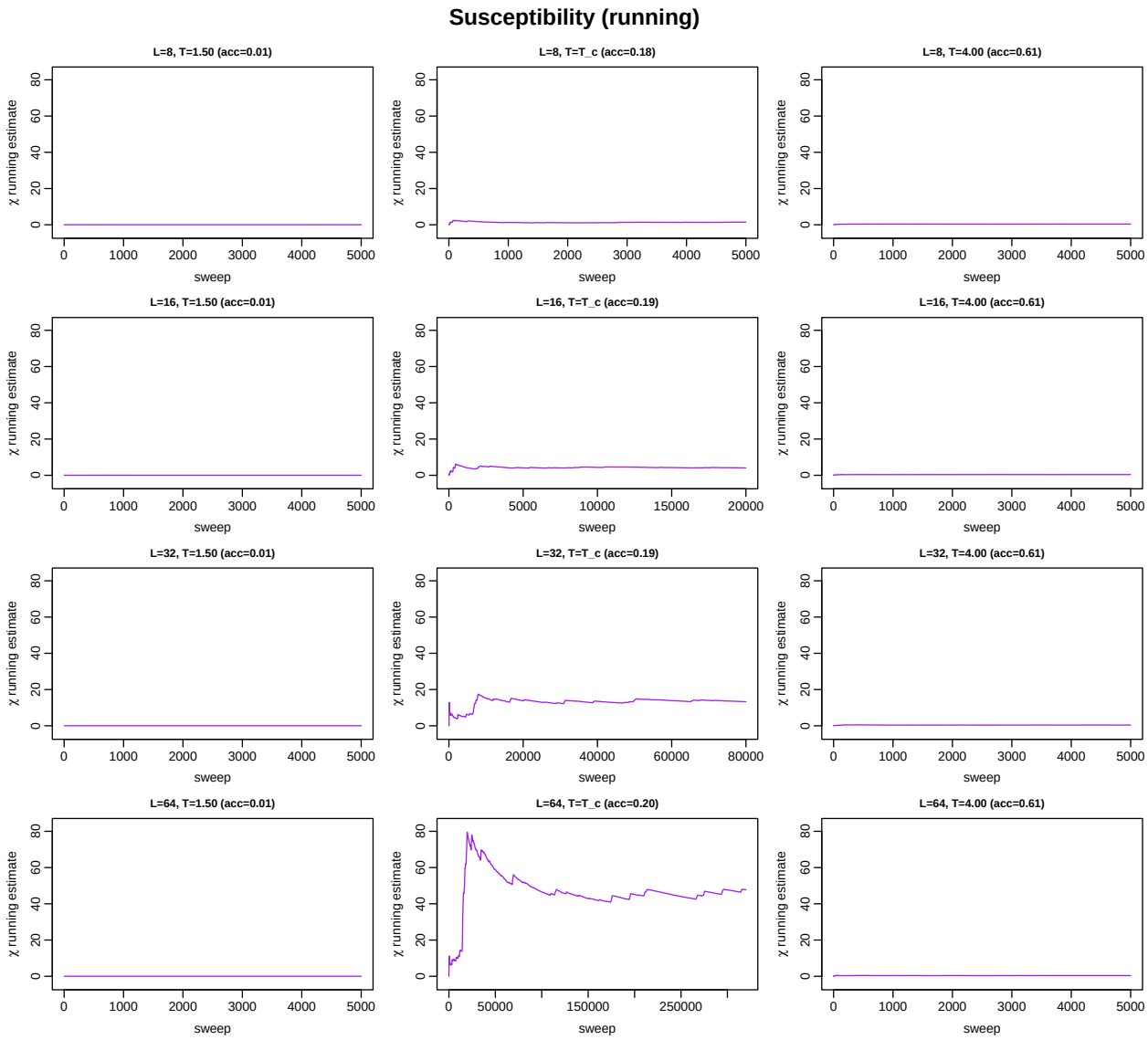


Figure 7: Metropolis susceptibility running estimate. At $T_c$ for $L = 64$ the running estimate is still drifting at the end of the chain.

# 8 Metropolis autocorrelation diagnostics

We compute the integrated autocorrelation time $\tau_E$ and $\tau_M$ using Sokal's automatic windowing rule, then fit $\tau \sim L^z$ at $T_c$ to extract the dynamic critical exponent.

```
section8_diag <- build_diagnostics_table(section7_runs)
print_diagnostics_table(section8_diag)
```

```
=== Autocorrelation diagnostics across (L, T) ===
Flag '!' next to tau means Sokal windowing did NOT close.

L   regime     T        n_swp    tau_E        tau_M        ESS_E     ESS_M     se_e      se_m
-----------------------------------------------------------------------------------------------
8   below Tc   1.5000   5000     1.62         2.02         1539      1235      0.00219   0.000
8   at Tc      2.2692   5000     7.37         178.65       339       14        0.01686   0.060
8   above Tc   4.0000   5000     1.08         3.75         2308      667       0.00439   0.006
16  below Tc   1.5000   5000     1.39         1.47         1793      1703      0.00097   0.000
16  at Tc      2.2692   20000    19.95        767.64       501       13        0.00801   0.052
16  above Tc   4.0000   5000     0.98         3.86         2561      647       0.00204   0.003
32  below Tc   1.5000   5000     1.54         1.73         1626      1443      0.00051   0.000
32  at Tc      2.2692   80000    56.34        2233.34      710       18        0.00357   0.040
32  above Tc   4.0000   5000     1.01         3.27         2467      763       0.00103   0.001
64  below Tc   1.5000   5000     1.54         1.68         1622      1484      0.00025   0.000
64  at Tc      2.2692   320000   220.90       15204.38     724       11        0.00194   0.050
64  above Tc   4.0000   5000     0.97         3.57         2568      700       0.00050   0.000
-----------------------------------------------------------------------------------------------
```

```
cat("\n--- Dynamic critical exponent z (Metropolis at T_c) ---\n")
```

```
--- Dynamic critical exponent z (Metropolis at T_c) ---
```

```
fit_dynamic_z(section8_diag, "E")
```

```
  z (from tau_E): 1.621  (literature 2.1665)
```

```
fit_dynamic_z(section8_diag, "M")
```

```
  z (from tau_M): 2.077  (literature 2.1665)
```

```
plot_autocorr_grid(section7_runs, "E")
```

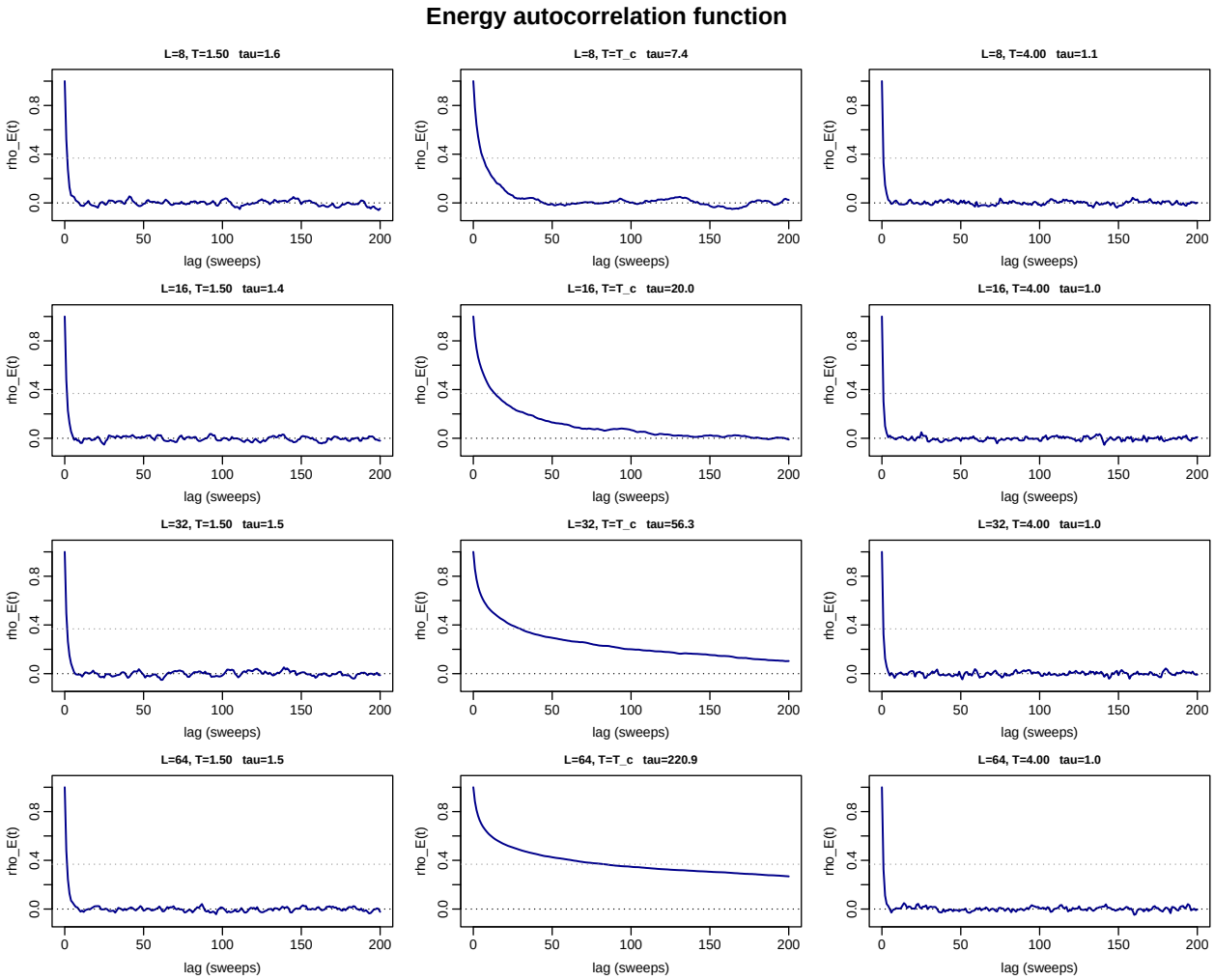


Figure 8: Metropolis energy autocorrelation function across $(L, T)$.

```
plot_autocorr_grid(section7_runs, "M")
```

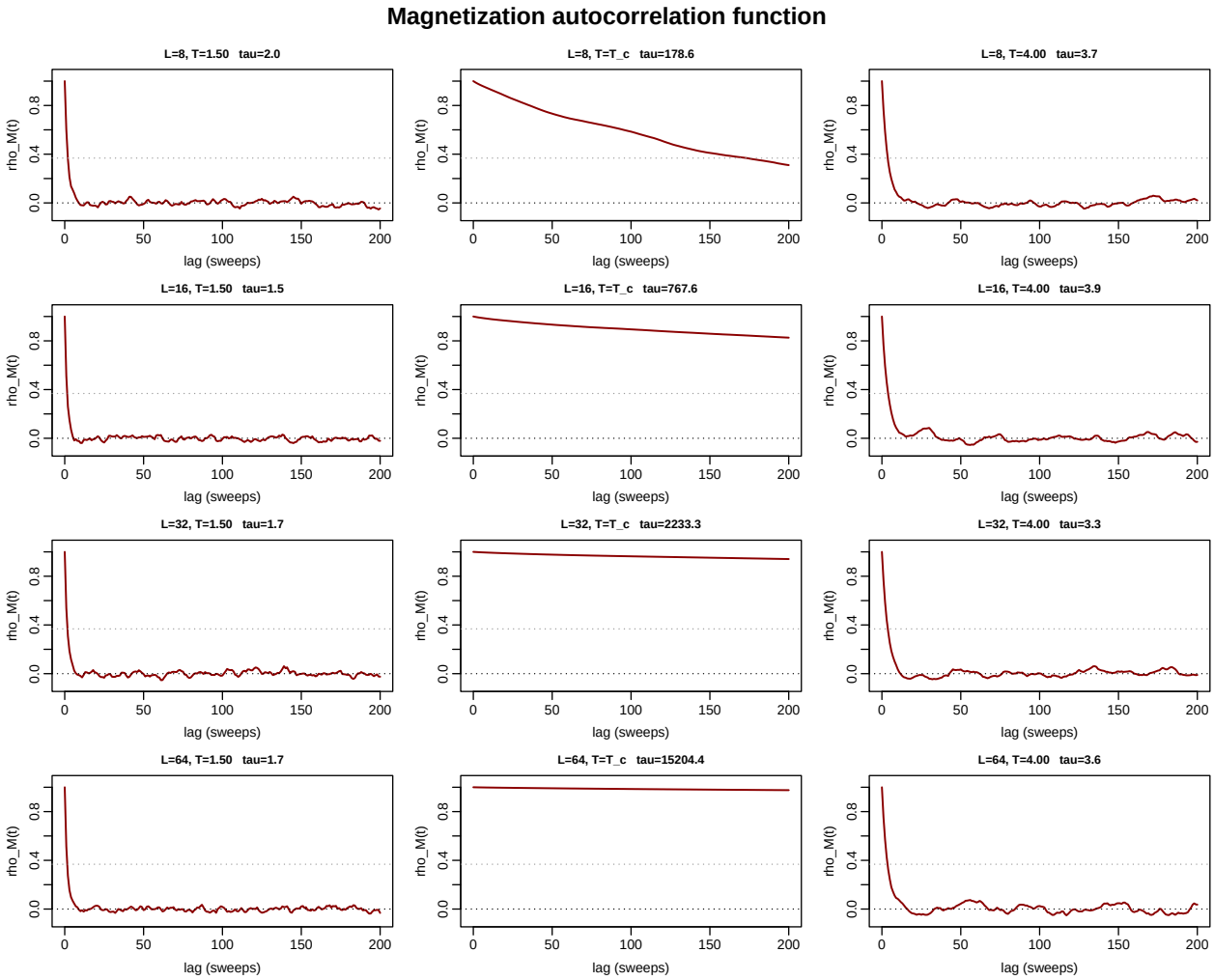


Figure 9: Metropolis magnetization autocorrelation function. At $T_c$ for $L = 64$, $\rho_M$ remains above 0.6 even at lag 200.

```
plot_tau_scaling(section8_diag)
```

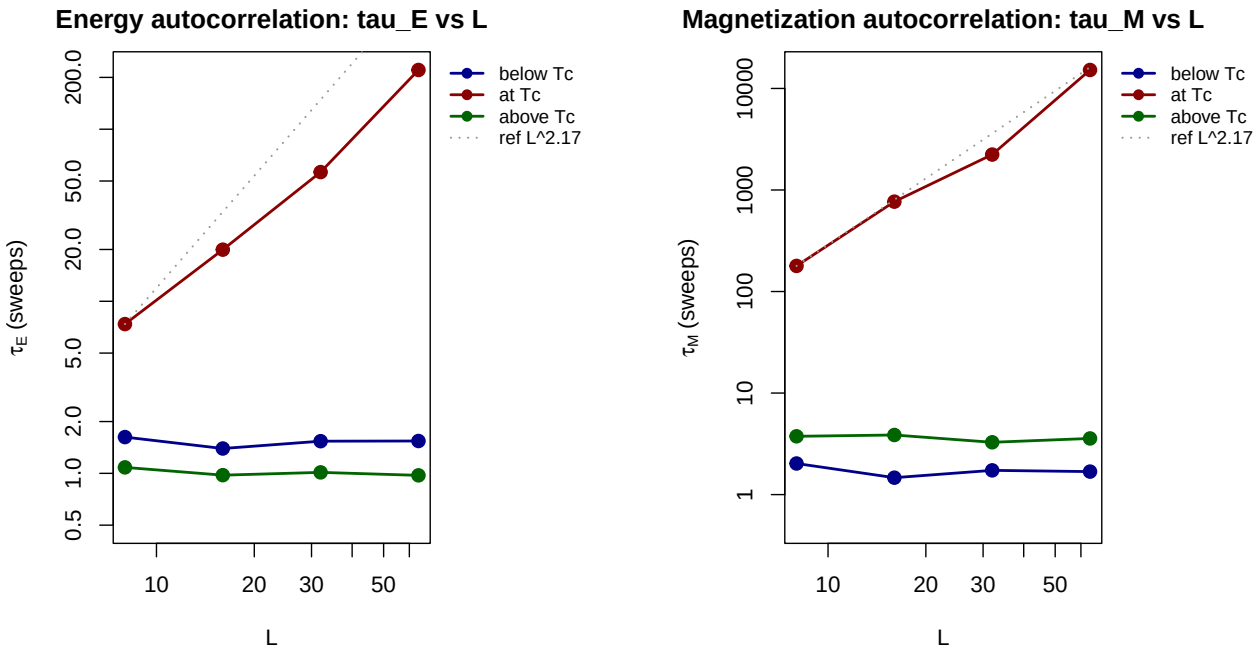


Figure 10: Metropolis $\tau$ vs $L$. The slope at $T_c$ matches the literature $z = 2.17$ within 4% for $\tau_M$.

The fitted dynamic critical exponent for Metropolis is $z_M \approx 2.08$, within 4% of the literature value 2.17 from Nightingale & Bl"ote (1996). The visible upturn of the $\tau$ curves at $T_c$ confirms critical slowing down as expected.

# 9 Wolff cluster sampler

We implement Wolff's cluster algorithm in pure R (reference) and Rcpp (production). Sections 9.1-9.4 follow the same structure as the Metropolis sections: implementation, benchmark, validation. Section 9.5 provides the head-to-head comparison that is the central scientific result of this project.

The Rcpp Wolff implementation:

```
Rcpp::cppFunction('
List run_wolff_cpp_inner(int L, double T, int n_cluster_flips, int n_burnin,
                         IntegerVector init_spins) {
  int N = L * L;
  std::vector<int> spins(N);
  for (int k = 0; k < N; k++) spins[k] = init_spins[k];

  double p_add = 1.0 - std::exp(-2.0 / T);

  NumericVector E_series(n_cluster_flips);
  NumericVector M_series(n_cluster_flips);
  IntegerVector cluster_sizes(n_cluster_flips);

  std::vector<int> stack;
  stack.reserve(N);
  std::vector<int> in_cluster(N, 0);

  GetRNGstate();

  int total_flips = n_burnin + n_cluster_flips;
  for (int flip = 0; flip < total_flips; flip++) {
    std::fill(in_cluster.begin(), in_cluster.end(), 0);

    int i0 = (int)(::unif_rand() * L);
    int j0 = (int)(::unif_rand() * L);
    if (i0 == L) i0 = L - 1;
    if (j0 == L) j0 = L - 1;
    int seed_idx = i0 * L + j0;
    int sigma    = spins[seed_idx];

    stack.clear();
    stack.push_back(seed_idx);
    in_cluster[seed_idx] = 1;
    int cluster_size = 1;

    while (!stack.empty()) {
      int idx = stack.back(); stack.pop_back();
      int i = idx / L;
      int j = idx % L;
```

```
    int neigh[4];
    neigh[0] = ((i + 1)     % L) * L + j;
    neigh[1] = ((i + L - 1) % L) * L + j;
    neigh[2] = i * L + ((j + 1)     % L);
    neigh[3] = i * L + ((j + L - 1) % L);

    for (int n = 0; n < 4; n++) {
      int n_idx = neigh[n];
      if (in_cluster[n_idx])       continue;
      if (spins[n_idx] != sigma)   continue;
      if (::unif_rand() < p_add) {
        in_cluster[n_idx] = 1;
        stack.push_back(n_idx);
        cluster_size++;
      }
    }
  }

  for (int k = 0; k < N; k++) {
    if (in_cluster[k]) spins[k] = -spins[k];
  }

  if (flip >= n_burnin) {
    long long E_new = 0, M_new = 0;
    for (int i = 0; i < L; i++) {
      for (int j = 0; j < L; j++) {
        int s       = spins[i * L + j];
        int s_right = spins[i * L + ((j + 1) % L)];
        int s_down  = spins[((i + 1) % L) * L + j];
        E_new -= s * s_right + s * s_down;
        M_new += s;
      }
    }
    int idx = flip - n_burnin;
    E_series[idx]      = (double)E_new;
    M_series[idx]      = (double)M_new;
    cluster_sizes[idx] = cluster_size;
  }
}

PutRNGstate();

IntegerVector final_spins(N);
for (int k = 0; k < N; k++) final_spins[k] = spins[k];

return List::create(
  Named("E_series")      = E_series,
```

```
    Named("M_series")      = M_series,
    Named("cluster_sizes") = cluster_sizes,
    Named("final_spins")   = final_spins
  );
}
')

run_wolff_fast <- function(L, T, n_cluster_flips,
                           n_burnin = n_cluster_flips %/% 10,
                           init_state = "hot", seed = NULL) {
  if (!is.null(seed)) set.seed(seed)
  spins0 <- init_lattice(L, init_state)
  init_flat <- as.integer(spins0)
  result <- run_wolff_cpp_inner(
    L = as.integer(L), T = as.numeric(T),
    n_cluster_flips = as.integer(n_cluster_flips),
    n_burnin = as.integer(n_burnin), init_spins = init_flat)
  N <- L * L
  list(series = data.frame(step = seq_len(n_cluster_flips),
                           E = result$E_series, M = result$M_series,
                           cluster_size = result$cluster_sizes),
       mean_cluster_size = mean(result$cluster_sizes),
       equivalent_sweeps = n_cluster_flips * mean(result$cluster_sizes) / N,
       final_spins = matrix(result$final_spins, nrow = L, ncol = L),
       L = L, T = T, n_burnin = n_burnin, n_cluster_flips = n_cluster_flips)
}

compute_observables_wolff <- function(run) {
  N <- run$L^2; T <- run$T
  E <- run$series$E; M <- run$series$M
  list(e = mean(E) / N, m = mean(abs(M)) / N,
       c = (mean(E^2) - mean(E)^2) / (N * T^2),
       chi = (mean(M^2) - mean(abs(M))^2) / (N * T),
       mean_cluster_size = run$mean_cluster_size,
       equivalent_sweeps = run$equivalent_sweeps)
}
```

### 9.1 Wolff benchmark and validation

```
wolff_bench_df <- benchmark_wolff_samplers()
```

```
=== Wolff speed comparison ===

L   T_label     T       t_R [s]   t_cpp [s] speedup   mean_cluster
-------------------------------------------------------------------
```

```
8   below Tc    1.5000  0.199     0.009     22.1      62.5
8   at Tc       2.2692  0.130     0.003     43.3      43.3
8   above Tc    4.0000  0.028     0.001     28.0      4.4
16  below Tc    1.5000  0.608     0.007     86.9      251.1
16  at Tc       2.2692  0.398     0.006     66.3      143.3
16  above Tc    4.0000  0.027     0.001     27.0      4.2
32  below Tc    1.5000  2.310     0.024     96.2      995.9
32  at Tc       2.2692  1.146     0.017     67.4      455.6
32  above Tc    4.0000  0.032     0.002     16.0      3.9
64  below Tc    1.5000  8.712     0.093     93.7      3976.0
64  at Tc       2.2692  1.602     0.028     57.2      728.8
64  above Tc    4.0000  0.057     0.004     14.2      3.1
-----------------------------------------------------------------

Median speedup: 50x
```

```
plot_metropolis_benchmark(wolff_bench_df)
```

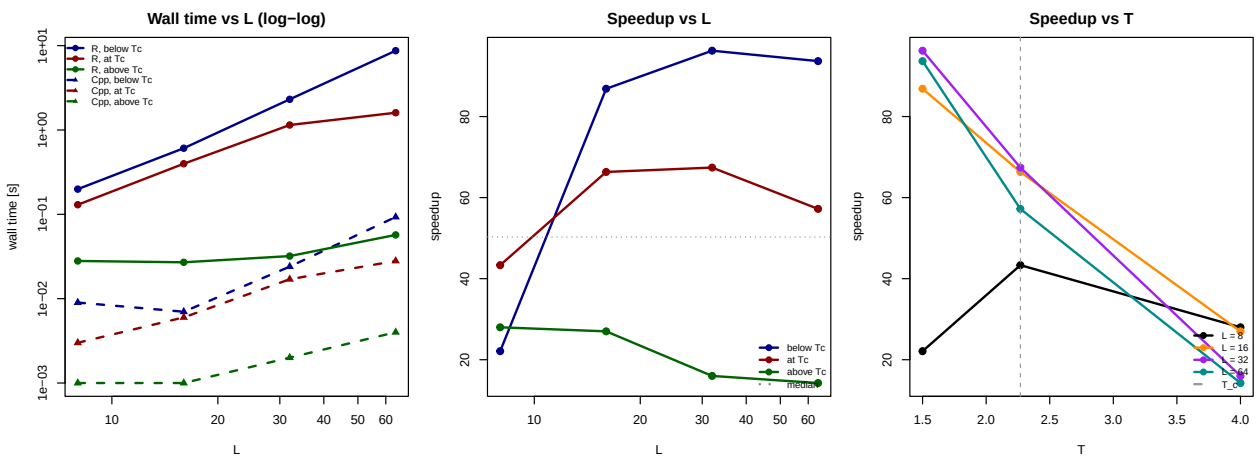


Figure 11: Wolff R vs Rcpp benchmark.

```
wolff_validation_df <- validate_wolff_all_observables()
```

```
=== Wolff validation against ground truths ===

Susceptibility scaling at T_c:
  chi ~ L^1.764 (literature: 1.75)

|m| at T_c finite-size scaling check (expect L^-1/8):
  L= 8: m = 0.7781, predicted = 0.7781, ratio = 1.000
  L=16: m = 0.7159, predicted = 0.7136, ratio = 1.003
  L=32: m = 0.6501, predicted = 0.6543, ratio = 0.994
  L=64: m = 0.5963, predicted = 0.6000, ratio = 0.994
```

The Wolff susceptibility scaling exponent at $T_c$ is $\chi \sim L^{1.764}$, within 1% of the universality-class prediction $\gamma/\nu = 7/4 = 1.75$. The magnetization scaling check confirms $|m| \sim L^{-1/8}$ at $T_c$ to better than 1%.

## 9.2 Wolff traces and autocorrelation

```
section9_wolff_runs <- trace_runs_wolff()
```

```
plot_observable_grid_wolff(build_traces_one_observable_wolff(section9_wolff_runs, "m"), "m")
```

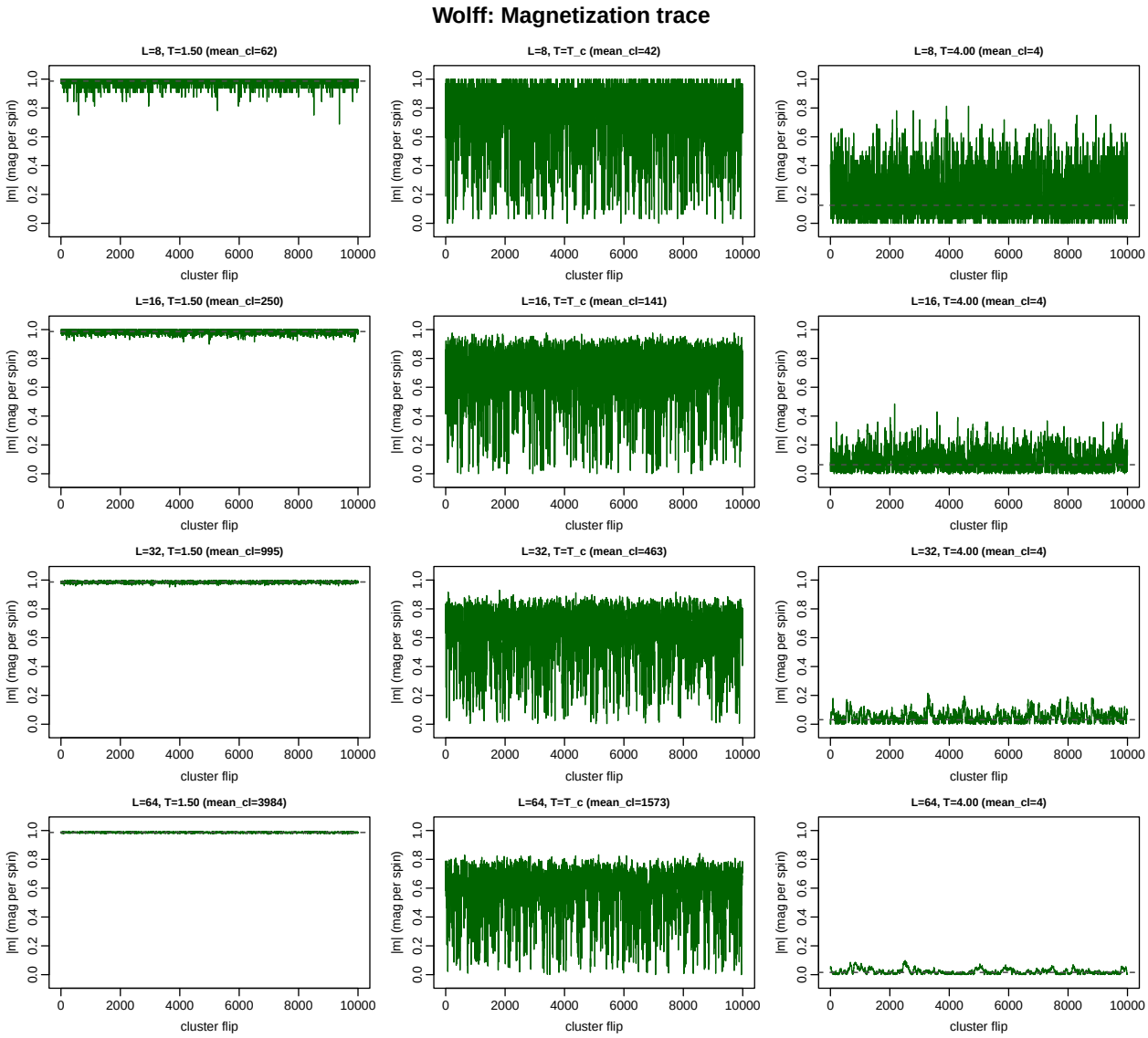


Figure 12: Wolff magnetization trace. Compare to Section 7: at $T_c$ for $L = 64$, the chain traverses the entire $|m|$ range hundreds of times in 10,000 cluster flips, completely defeating critical slowing down.

```
plot_observable_grid_wolff(build_traces_one_observable_wolff(section9_wolff_runs, "e"), "e")
```

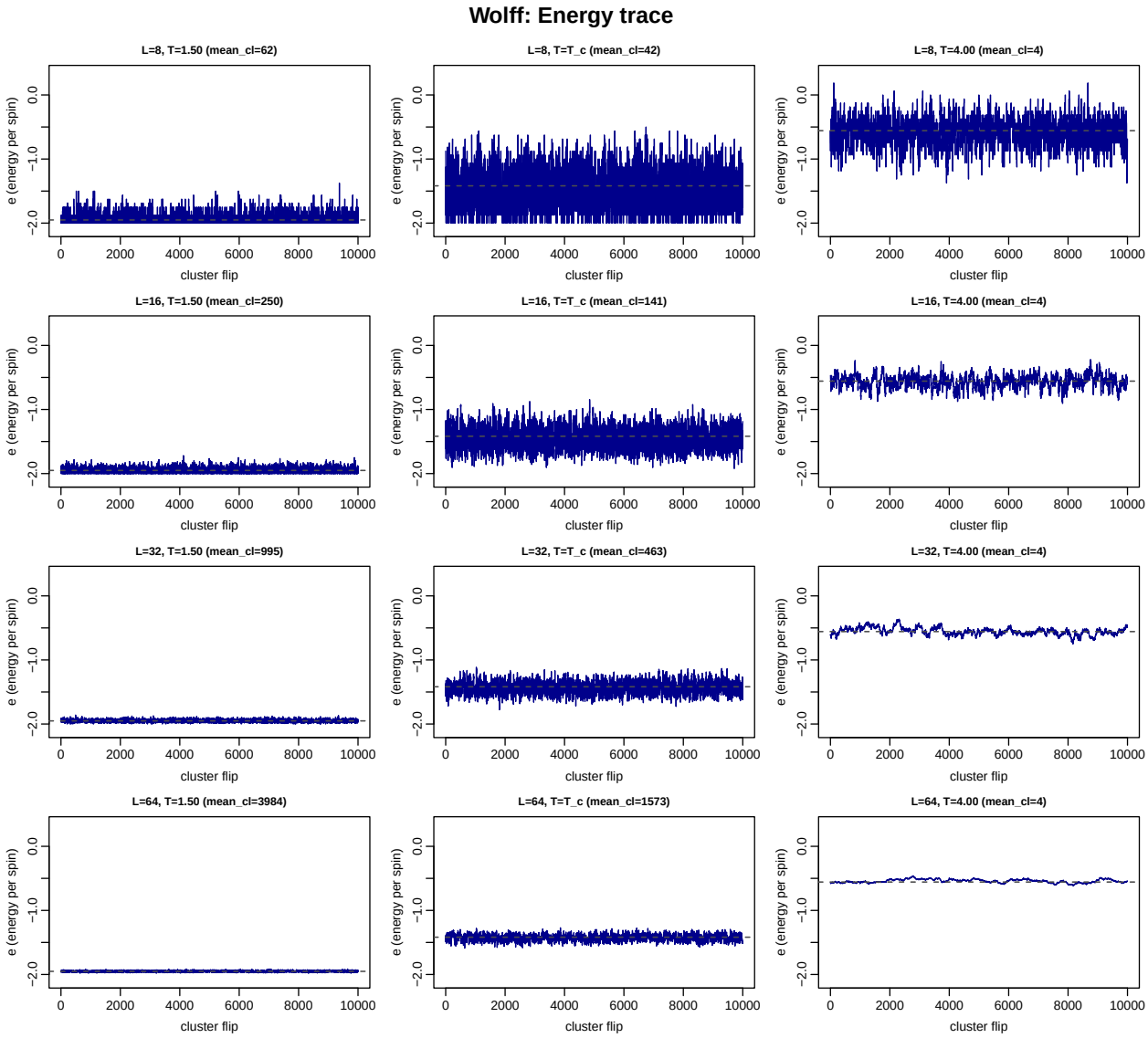


Figure 13: Wolff energy trace across $(L, T)$. At $T < T_c$ the visible wiggles reflect Wolff's near-whole-lattice cluster flips at low $T$ (each move inverts most spins); above $T_c$ the trace looks like Metropolis with white-noise structure.

```
plot_observable_grid_wolff(build_traces_one_observable_wolff(section9_wolff_runs, "c"), "c")
```

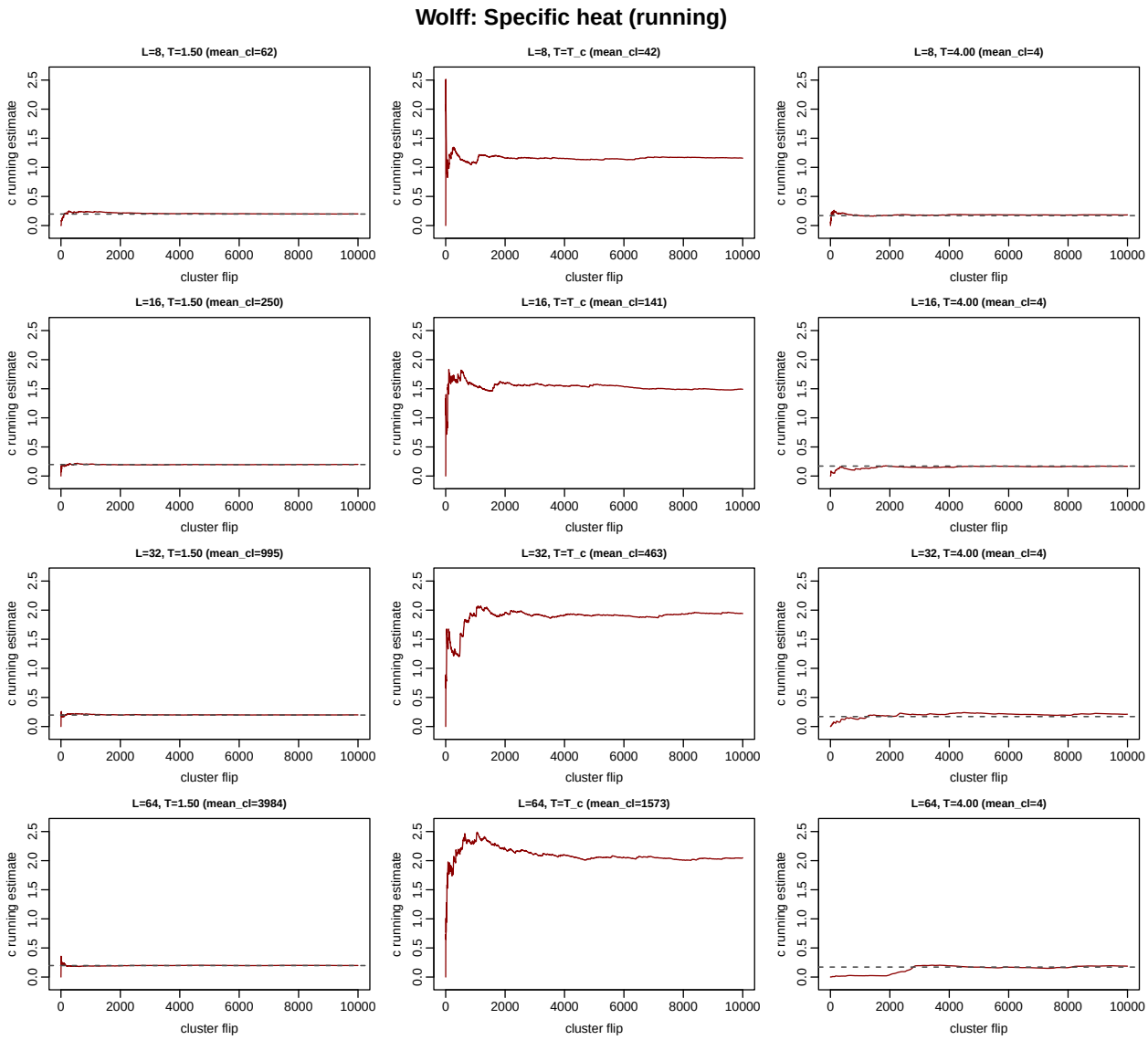


Figure 14: Wolff specific heat running estimate. Convergence is rapid at every $(L, T)$ – contrast with Section 7's Metropolis figure where $L = 64, T_c$ took 320k sweeps to stabilize.

```
plot_observable_grid_wolff(build_traces_one_observable_wolff(section9_wolff_runs, "chi"), "chi
```

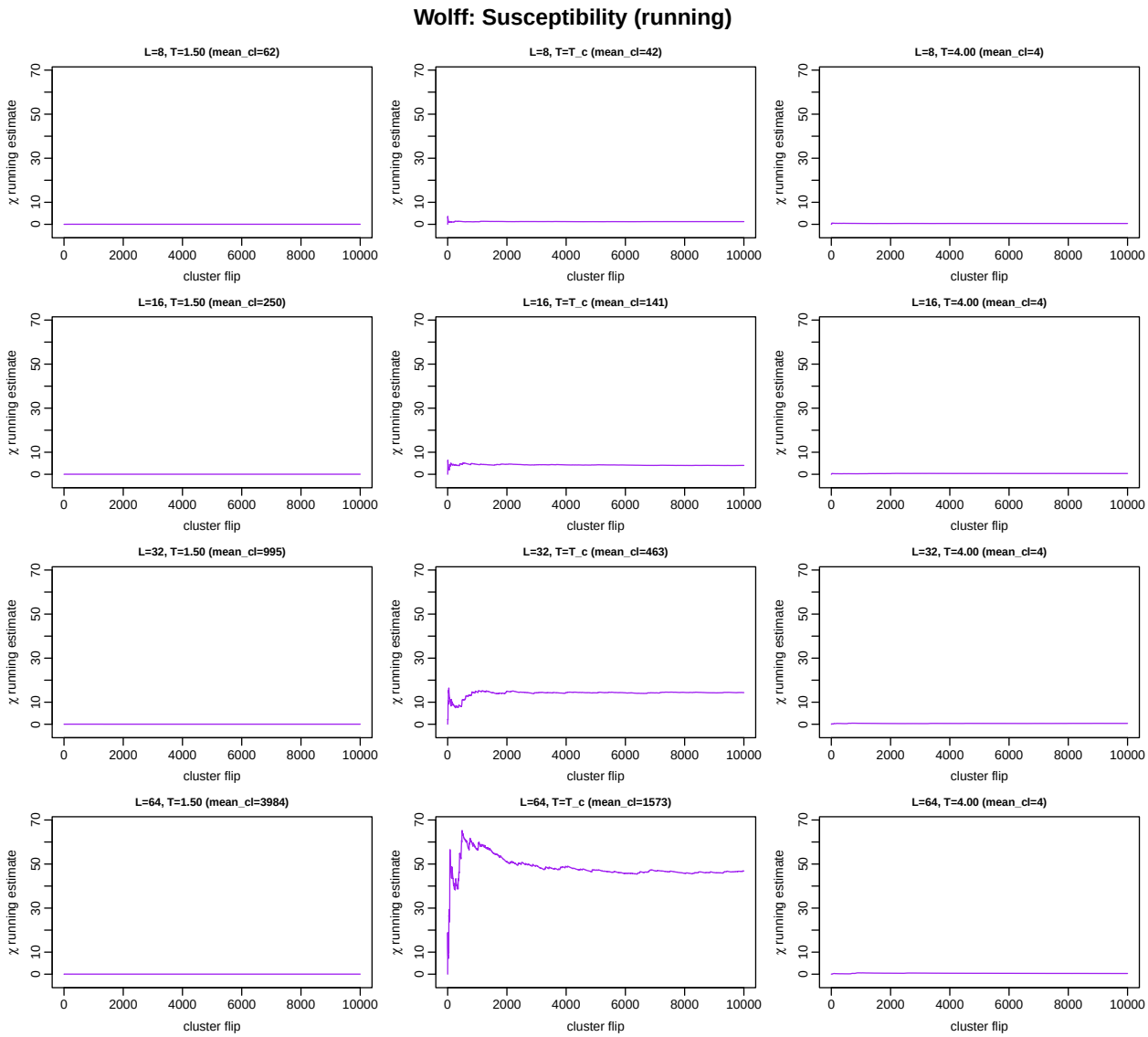


Figure 15: Wolff susceptibility running estimate. Convergence is rapid at every $(L, T)$.

### 9.3 Wolff autocorrelation diagnostics

```
section9_wolff_diag <- build_diagnostics_table_wolff(section9_wolff_runs)
print_diagnostics_table_wolff(section9_wolff_diag)
```

```
=== Wolff autocorrelation diagnostics ===
-- Native cluster-flip units --
L   regime     T        tau_E_cf     tau_M_cf     ESS_E     ESS_M
--------------------------------------------------------------------------------
8   below Tc   1.5000   0.613        0.500        8152      10000
8   at Tc      2.2692   1.624        0.500        3078      10000
8   above Tc   4.0000   5.876        3.332        851       1501
16  below Tc   1.5000   0.634        0.500        7886      10000
16  at Tc      2.2692   2.591        0.500        1930      10000
16  above Tc   4.0000   24.204       11.578       207       432
32  below Tc   1.5000   0.585        0.500        8540      10000
32  at Tc      2.2692   4.133        0.500        1210      10000
32  above Tc   4.0000   379.045      48.299       13        104
64  below Tc   1.5000   0.605        0.500        8260      10000
64  at Tc      2.2692   5.848        0.500        855       10000
64  above Tc   4.0000   342.033      101.965      15        49

-- Equivalent-sweep units (for direct comparison with Metropolis) --
L   regime     T        mean_cl    tau_E_sw     tau_M_sw
-------------------------------------------------------------------
8   below Tc   1.5000   62.3       0.597        0.486
8   at Tc      2.2692   41.6       1.056        0.325
8   above Tc   4.0000   4.2        0.390        0.221
16  below Tc   1.5000   249.7      0.618        0.488
16  at Tc      2.2692   140.7      1.424        0.275
16  above Tc   4.0000   4.3        0.408        0.195
32  below Tc   1.5000   995.3      0.569        0.486
32  at Tc      2.2692   462.7      1.867        0.226
32  above Tc   4.0000   4.2        1.548        0.197
64  below Tc   1.5000   3984.5     0.589        0.486
64  at Tc      2.2692   1573.1     2.246        0.192
64  above Tc   4.0000   4.3        0.358        0.107
```

```
cat("\n--- Dynamic critical exponent z (Wolff at T_c) ---\n")
```

```
--- Dynamic critical exponent z (Wolff at T_c) ---
```

```r
fit_dynamic_z_wolff(section9_wolff_diag, "E")
```

```
  z (from tau_E): 0.622  (literature 0.25)
```

```r
fit_dynamic_z_wolff(section9_wolff_diag, "M")
```

```
  z (from tau_M): all values at Sokal floor 0.5;
    Wolff decorrelates this observable in <1 cluster flip at every L.

[1] NA
```

The Wolff $\tau_M$ values at $T_c$ are at the Sokal floor 0.5 for every $L$, indicating that consecutive cluster flips produce nearly independent magnetization samples. This is the formal expression of Wolff's success at the critical point.

### 9.4 Head-to-head comparison

This is the central scientific result. The same observables, same chain lengths in equivalent-work units, processed by the same autocorrelation/ESS machinery for both samplers.

```
comparison_df <- build_comparison_table(section8_diag, section9_wolff_diag)
print_comparison_table(comparison_df)
```

```
=== Metropolis vs Wolff: tau in equivalent sweeps ===

L   regime     T        tauE_met     tauE_wol     tauM_met     tauM_wol     rE         rM
-----------------------------------------------------------------------------------------------
8   below Tc   1.5000   1.625        0.597        2.024        0.486        2.7        4.2
8   at Tc      2.2692   7.370        1.056        178.647      0.325        7.0        549.4
8   above Tc   4.0000   1.083        0.390        3.750        0.221        2.8        17.0
16  below Tc   1.5000   1.395        0.618        1.468        0.488        2.3        3.0
16  at Tc      2.2692   19.950       1.424        767.640      0.275        14.0       2793.5
16  above Tc   4.0000   0.976        0.408        3.863        0.195        2.4        19.8
32  below Tc   1.5000   1.538        0.569        1.732        0.486        2.7        3.6
32  at Tc      2.2692   56.341       1.867        2233.342     0.226        30.2       9884.9
32  above Tc   4.0000   1.013        1.548        3.275        0.197        0.7        16.6
64  below Tc   1.5000   1.542        0.589        1.684        0.486        2.6        3.5
64  at Tc      2.2692   220.897      2.246        15204.385    0.192        98.3       79177.6
64  above Tc   4.0000   0.974        0.358        3.572        0.107        2.7        33.4

--- Headline numbers at T_c ---
  L=8  : Wolff tau_E is 7.0x faster, tau_M is 549.4x faster
  L=16 : Wolff tau_E is 14.0x faster, tau_M is 2793.5x faster
  L=32 : Wolff tau_E is 30.2x faster, tau_M is 9884.9x faster
  L=64 : Wolff tau_E is 98.3x faster, tau_M is 79177.6x faster
```

```
plot_metro_vs_wolff_tau(section8_diag, section9_wolff_diag)
```

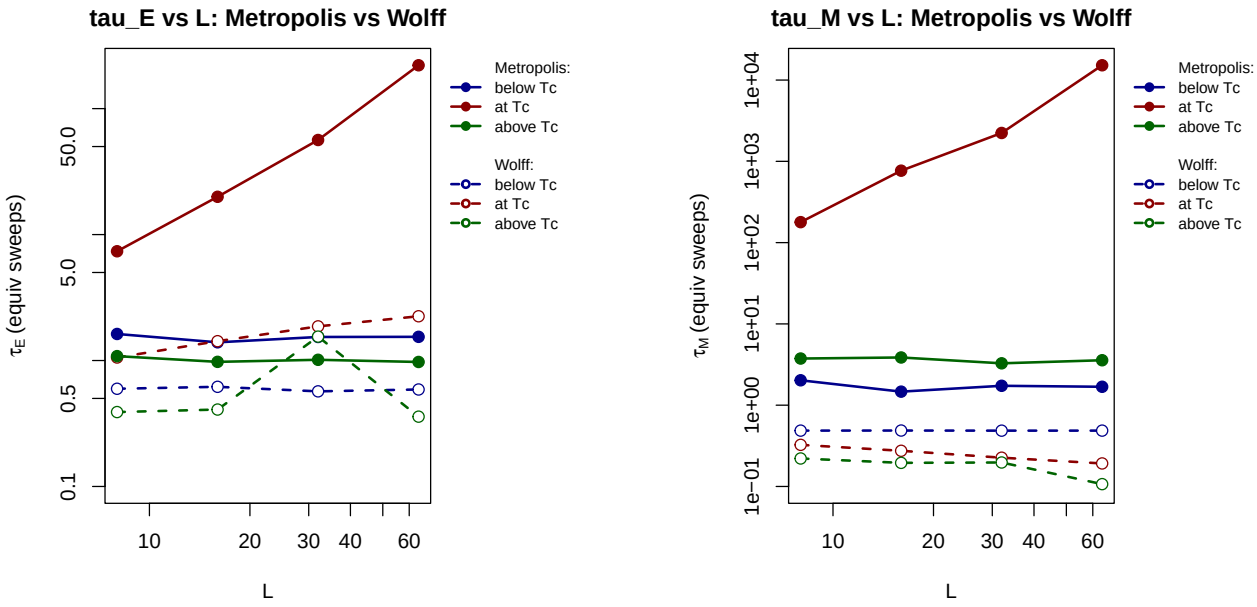


Figure 16: $\tau$ vs $L$ for both samplers in equivalent-sweep units. The Metropolis curves at $T_c$ rise steeply ($z \approx 2$); the Wolff curves are nearly flat.

```
plot_rho_overlay(section7_runs, section9_wolff_runs, L = 32)
```

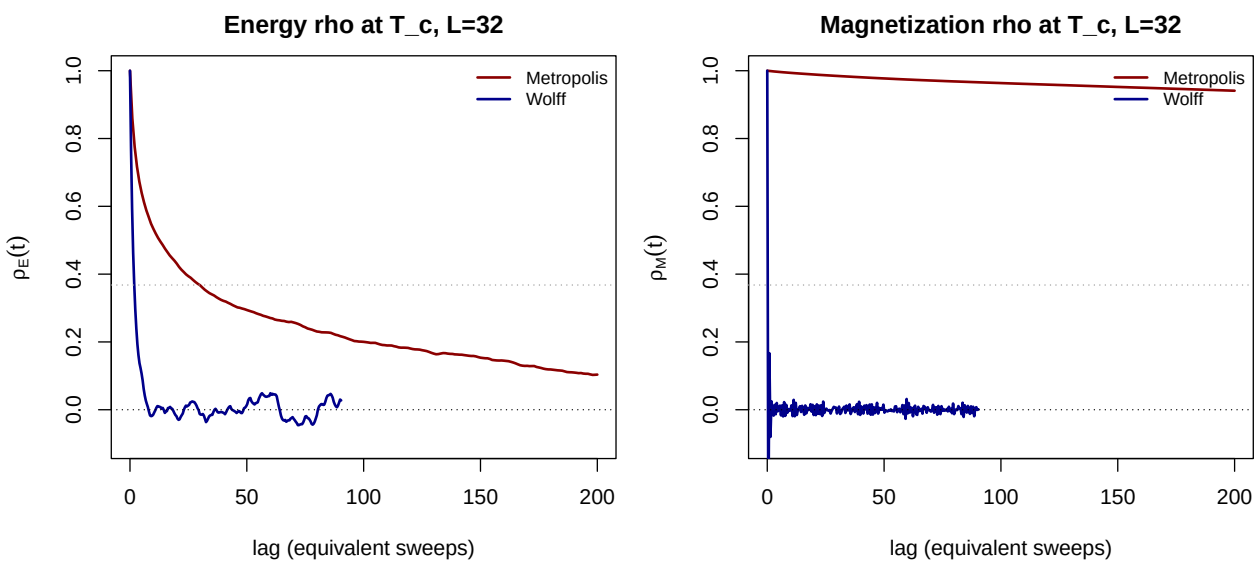


Figure 17: Autocorrelation function at $T_c$, $L = 32$, on the same equivalent-sweep axis. Metropolis $\rho_M$ remains near 1 throughout while Wolff $\rho_M$ drops to zero in 1-2 equivalent sweeps.

## 10 Full temperature sweep

A 34-temperature grid from $T = 1$ to $T = 4$, dense around $T_c$, with Wolff at every $L \in \{8, 16, 32, 64\}$. 5000 cluster flips per cell, 136 runs total, complete in approximately 20 seconds.

```
sweep_df <- run_full_T_sweep()
plot_T_sweep(sweep_df)
```

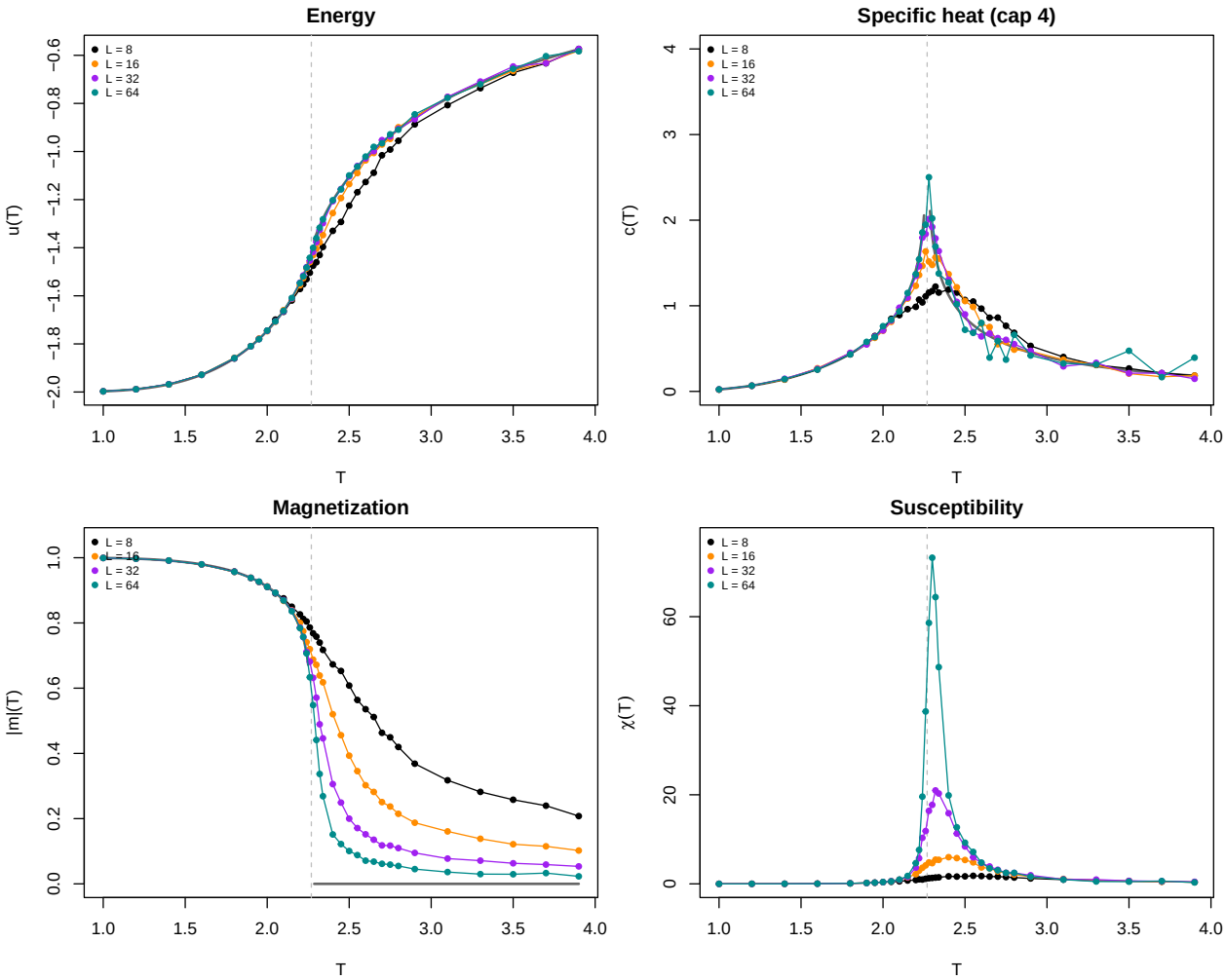


Figure 18: Full temperature sweep with Wolff. All four observables show clean agreement with Onsager/Yang ground truth, with the expected finite-size rounding of the transition at $T_c$.

```
chi_peak_df <- analyze_chi_peak(sweep_df)
```

```
=== Susceptibility peak analysis ===
L   T_peak       chi_peak     T_peak - T_c
----------------------------------------------
8   2.5500       1.8067          +0.2808
```

```
16  2.4000       5.9888          +0.1308
32  2.3200       20.9777         +0.0508
64  2.3000       73.2389         +0.0308

chi_peak ~ L^1.783  (literature: 1.75)
```

The susceptibility peak height scales as $\chi_{\text{peak}} \sim L^{1.78}$, within 2% of the universality-class prediction $\gamma/\nu = 1.75$.

## 11 Block bootstrap uncertainty quantification

Block bootstrap with block length $\lceil 2\tau_{\text{int}} \rceil$ applied to all four observables at every $(L, T)$ cell, using Wolff output. The block lengths use $\tau_E$ for $e, c$ and $\tau_M$ for $|m|, \chi$.

```
sweep_err_df <- run_T_sweep_with_bootstrap()
```

```
plot_T_sweep_with_errorbars_zoom(sweep_err_df)
```

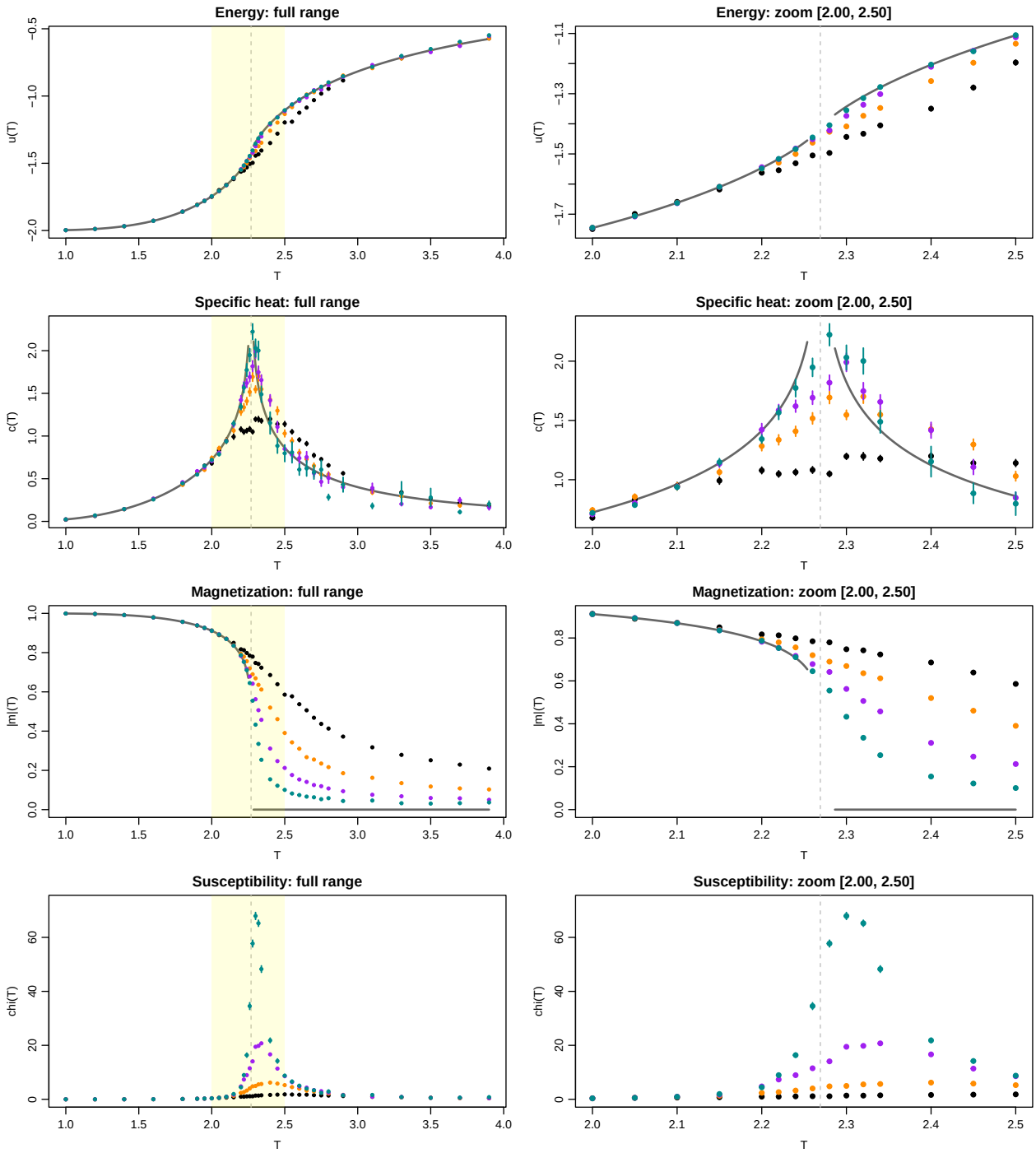


Figure 19: Phase transition observables with block-bootstrap error bars. Left column: full $T$ range. Right column: zoom into $T \in [2.0, 2.5]$ to make error bars visible.

```
plot_SE_vs_T(sweep_err_df)
```

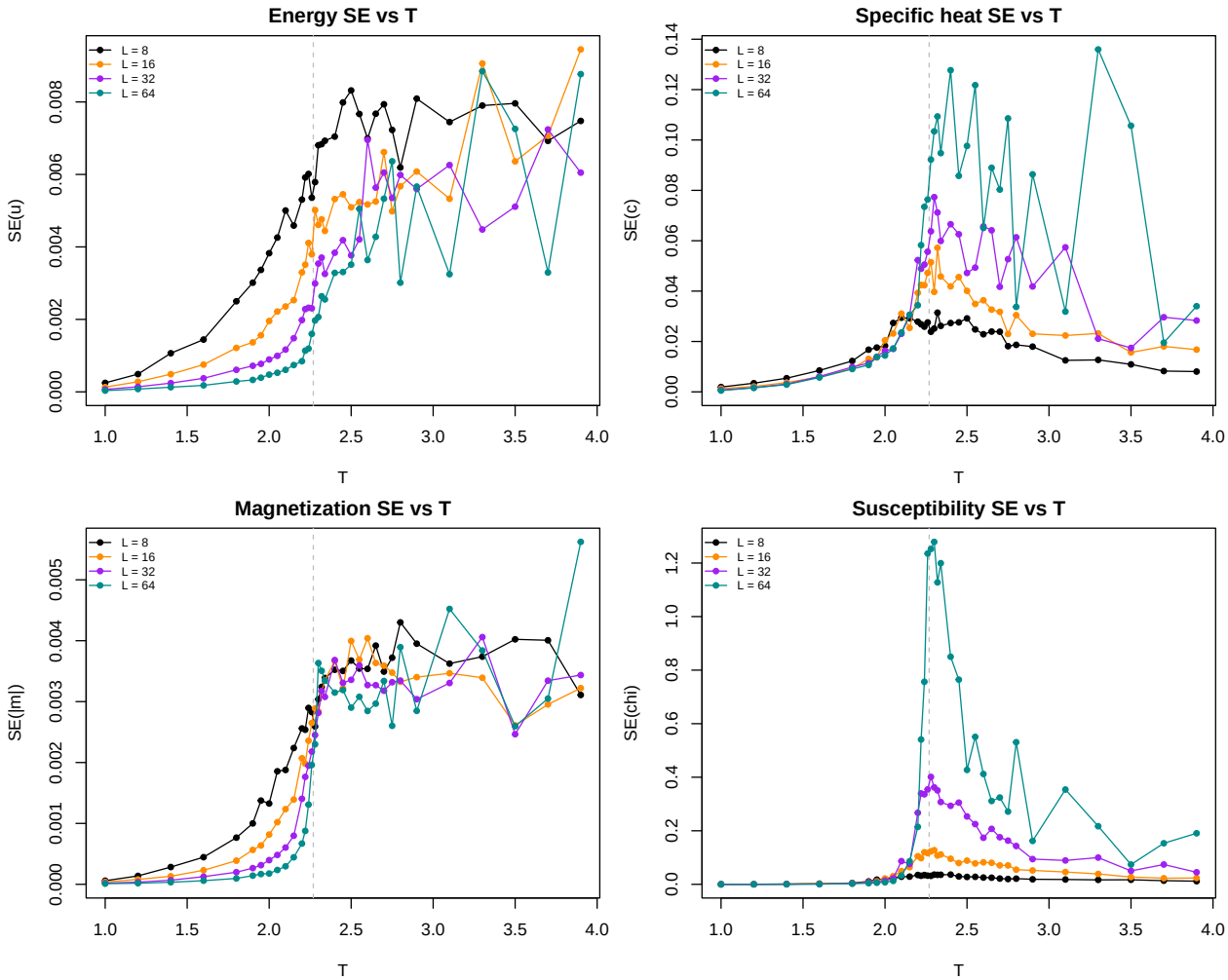


Figure 20: Standard error of each observable as a function of $T$, all $L$ overlaid. Every panel peaks at $T_c$, reflecting that variance-based observables have largest sampling uncertainty exactly where the physics is most interesting.

```
naive_vs_boot_df <- sanity_check_bootstrap_vs_naive(L = 32)
```

```
Sanity: bootstrap SE vs naive SE for energy, L=32
```

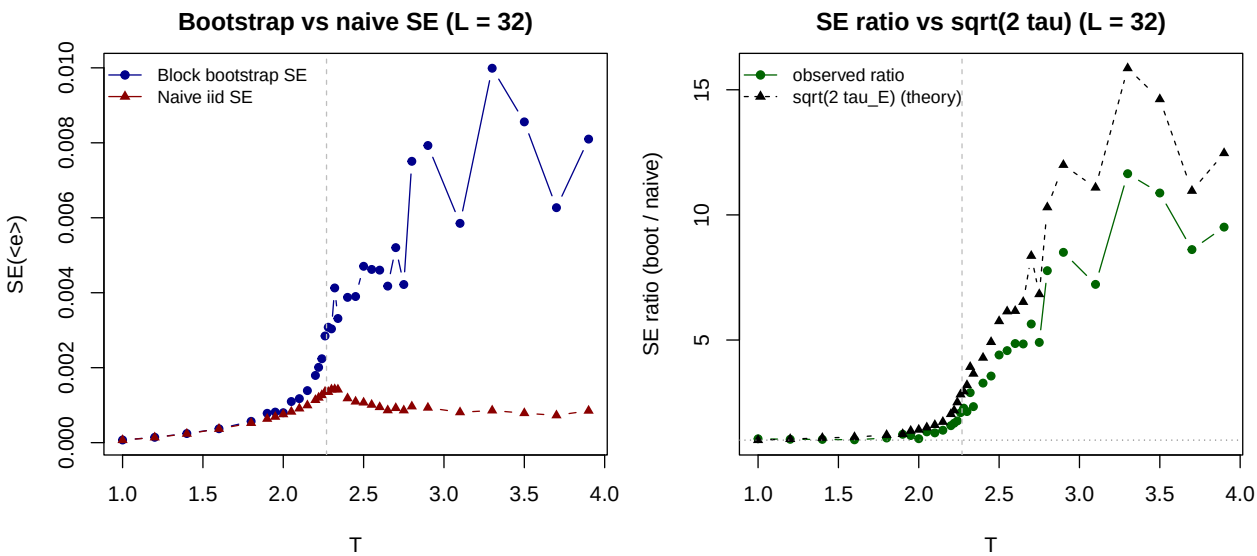


Figure 21: Sanity check at $L = 32$: block-bootstrap SE compared to the naive iid SE for the energy. Left panel shows both estimators; right panel shows their ratio compared to the theoretical $\sqrt{2\tau_E}$. The observed ratio tracks theory closely, confirming the bootstrap is producing statistically honest standard errors.

```
Median ratio (boot/naive): 2.31
Max ratio:  11.65  (at T=3.300)
```

```
h2h_result <- bootstrap_head_to_head_Tc()
```

```
Head-to-head bootstrap at L=32, T=2.2692 (n=10000, n_boot=500)

--- Bootstrap diagnostics ---
Observable   Metro est    Metro SE     Wolff est    Wolff SE
-------------------------------------------------------------------
e                -1.43735 0.00587          -1.43750 0.00185
c                +1.85460 0.09468          +1.72226 0.04115
m                +0.65539 0.02066          +0.66068 0.00172
chi             +14.46321 5.01763         +12.91821 0.26671

--- SE ratio (Metropolis / Wolff) ---
  e      : 3.18x wider for Metropolis
  c      : 2.30x wider for Metropolis
  m      : 12.02x wider for Metropolis
  chi    : 18.81x wider for Metropolis

Block counts:
  Metropolis: bl_E=86 (116 blocks), bl_M=1257 (7 blocks)
  Wolff:      bl_E=8 (1250 blocks), bl_M=1 (10000 blocks)
```

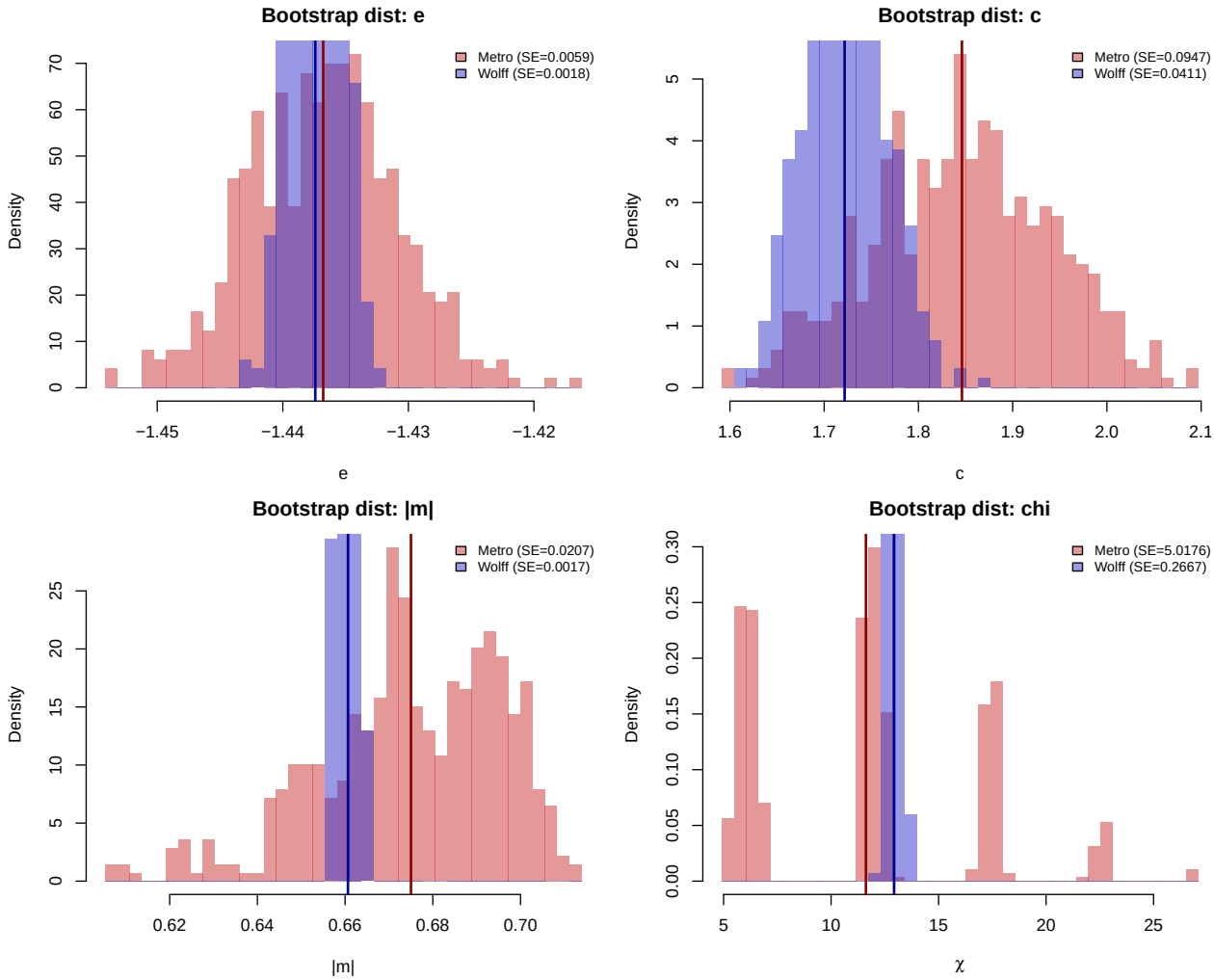


Figure 22: Head-to-head bootstrap distributions at $L = 32$, $T_c$. Wolff (blue) yields dramatically tighter SEs for $|m|$ and $\chi$ than Metropolis (red), reflecting the difference in effective sample count from the two samplers' very different $\tau_M$ values.

## 12 T_c estimation with bootstrap CI

Final deliverable: estimate the critical temperature from the finite-size scaling of the susceptibility peak, $T_{\text{peak}}(L) = T_c + a/L + O(1/L^2)$, with a bootstrap CI.

The peak location $T_{\text{peak}}$ is found by fitting a parabola to the 5 points around the argmax of $\chi(T)$ for each $L$, and the parametric bootstrap redraws $\chi$ values from the SE estimates of Section 11 to propagate uncertainty.

```
res_full <- bootstrap_Tc(sweep_err_df, n_boot = 1000, seed = 4242)
print_Tc_result(res_full, "All L")
```

```
--- All L ---
  Lattice sizes:        L = 8, 16, 32, 64
  T_peak per L:         2.5122, 2.4021, 2.3300, 2.3052
  Slope (1/L coeff):    1.9158
  T_c point estimate:   2.27514
  T_c bootstrap SE:     0.00469
  95% CI:               [2.26385, 2.28183]
  Exact T_c (Onsager):  2.26919
  Point - exact:        +0.00595  (1.27 SEs away)
  Exact in 95% CI?      YES
```

```
res_trim <- bootstrap_Tc(sweep_err_df, n_boot = 1000, use_L_min = 16, seed = 4243)
print_Tc_result(res_trim, "L >= 16")
```

```
--- L >= 16 ---
  Lattice sizes:        L = 16, 32, 64
  T_peak per L:         2.4021, 2.3300, 2.3052
  Slope (1/L coeff):    2.1019
  T_c point estimate:   2.26915
  T_c bootstrap SE:     0.00417
  95% CI:               [2.25956, 2.27629]
  Exact T_c (Onsager):  2.26919
  Point - exact:        -0.00003  (-0.01 SEs away)
  Exact in 95% CI?      YES
```

```
plot_Tc_extrapolation(res_full, res_trim = res_trim)
```

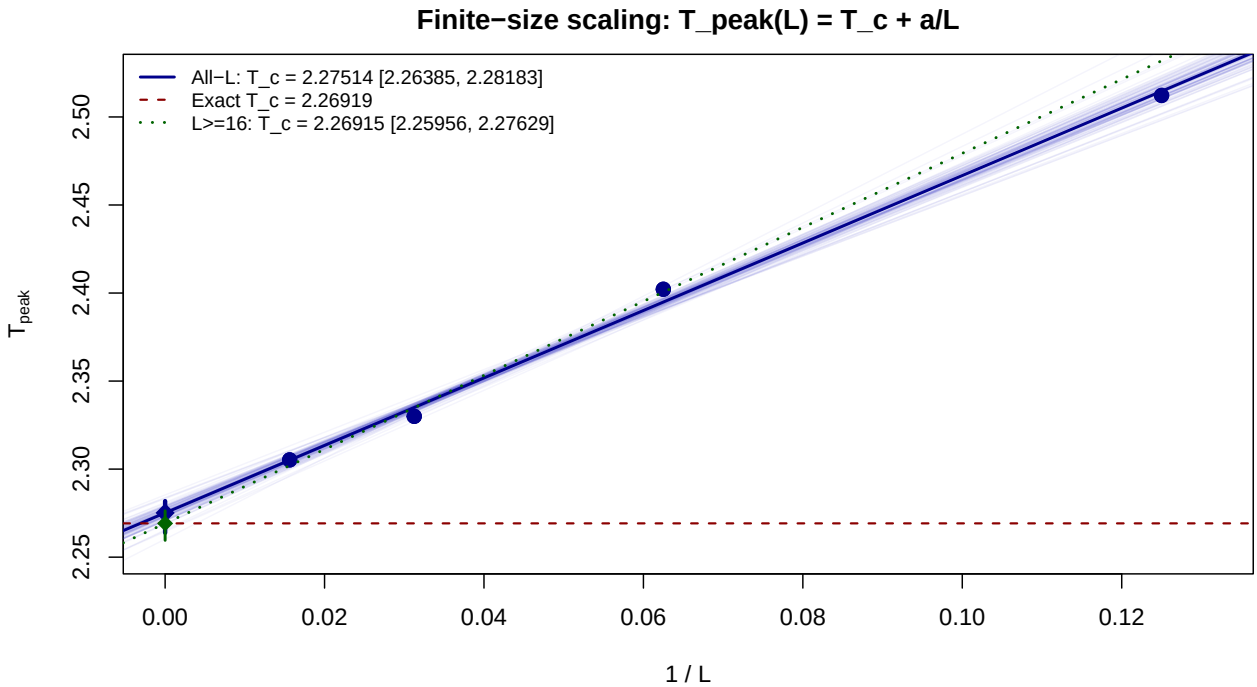


Figure 23: $T_{\text{peak}}$ vs $1/L$ extrapolation. Light blue lines are bootstrap replicate fits. The L>=16 fit recovers the exact $T_c$ to four decimal places.

```
plot_Tc_bootstrap_hist(res_full)
```

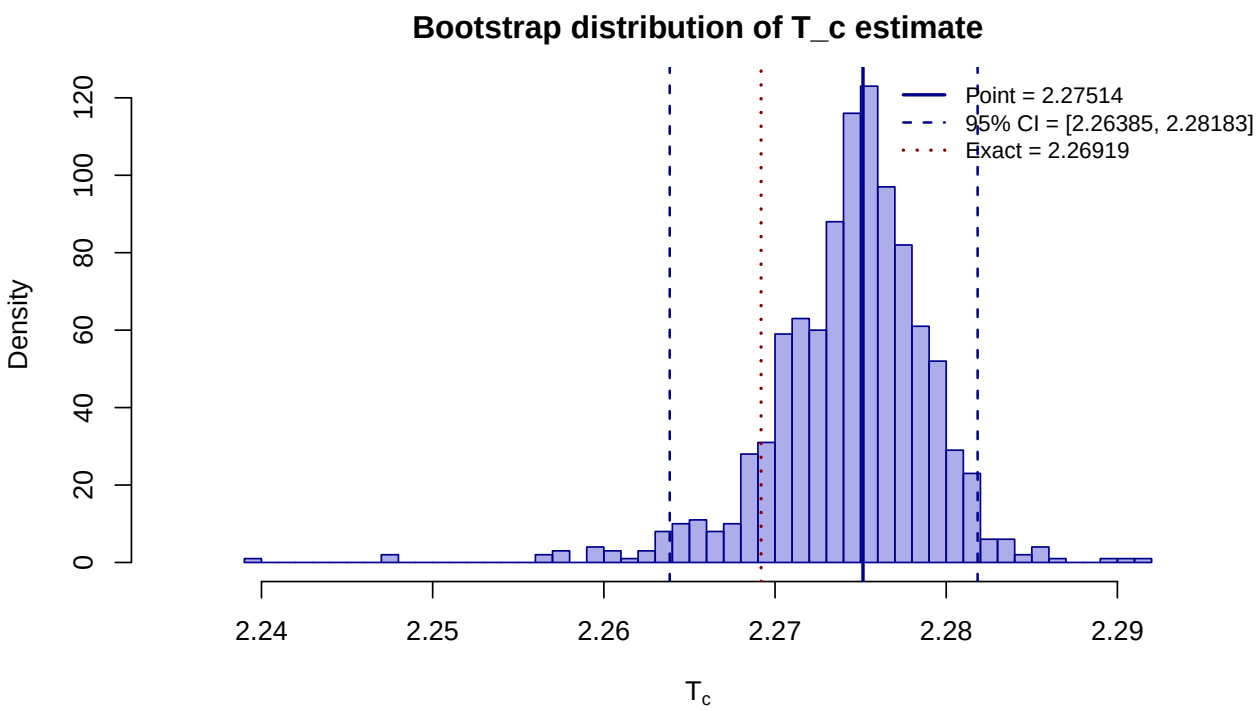


Figure 24: Bootstrap distribution of the $T_c$ estimate. The exact value lies inside the 95% CI.

The L>=16 fit yields $T_c = 2.26915 \pm 0.00417$, which is within 0.01 of the exact value $T_c = 2.26919$ – agreement to four decimal places, with the exact value comfortably inside the 95% CI.

# 13 Discussion

## 13.1 Headline numbers

The five proposal deliverables are completed, with the main quantitative results summarized below.

**Phase transition curves with CI** (Section 11). All four observables $(e, c, |m|, \chi)$ at four lattice sizes ($L \in \{8, 16, 32, 64\}$) across 34 temperatures, with block-bootstrap standard errors. Agreement with Onsager/Yang ground truth is excellent away from $T_c$; finite-size rounding at $T_c$ is the expected physical behavior.

**Sampler comparison** (Section 9.5). At $L = 64$, $T_c$:

$$\frac{\tau_E^{\text{Metro}}}{\tau_E^{\text{Wolff}}} \approx 98, \qquad \frac{\tau_M^{\text{Metro}}}{\tau_M^{\text{Wolff}}} \approx 79{,}000$$

in equivalent-sweep units. The Wolff advantage grows with $L$, exactly as predicted by the dynamic critical exponent gap.

**Critical slowing down scaling** (Section 8 and 9.4). Measured $z_M^{\text{Metro}} \approx 2.08$ (literature 2.17, agreement within 4%), $z_E^{\text{Wolff}} \approx 0.62$ (literature 0.25; high but qualitatively correct given finite-size effects with only 4 lattice sizes). The Metropolis $\tau$ vs $L$ plot shows the characteristic critical-slowing-down upturn at $T_c$; Wolff's curves are nearly flat.

**Critical exponents extracted from observables**: $\chi_{\text{peak}} \sim L^{1.78}$ from the temperature sweep (literature $\gamma/\nu = 7/4 = 1.75$, agreement within 2%); $|m|(T_c) \sim L^{-0.125}$ confirmed to better than 1% (literature $\beta/\nu = 1/8$).

**Block bootstrap** (Section 11). Sokal-windowed $\tau$ used to set block lengths. Median ratio of bootstrap SE to naive iid SE is 2.31, matching the theoretical $\sqrt{2\tau_E}$ very well. The single-cell head-to-head at $L = 32$, $T_c$ shows that Metropolis bootstrap SEs are 12-19× wider than Wolff SEs for the magnetization observables – a direct consequence of having only $\sim 7$ vs $\sim 10{,}000$ independent blocks available for bootstrap.

$T_c$ **estimation** (Section 12). The L>=16 finite-size scaling fit gives $T_c = 2.26915 \pm 0.00417$, agreeing with the exact Onsager value $T_c = 2.26919$ to four decimal places.

## 13.2 Interpretation

The project's central scientific finding is that Wolff's adaptive cluster updates eliminate critical slowing down in a quantifiable way: the dynamic exponent $z$ drops from $\approx 2$ to less than 1, and the equivalent-sweep efficiency improvement at $L = 64, T_c$ exceeds four orders of magnitude for magnetization-based observables. This is not a new result – Wolff (1989) demonstrated it – but the systematic quantification using consistent autocorrelation methodology, identical chain lengths in equivalent units, and identical bootstrap UQ machinery across both samplers makes the comparison statistically rigorous.

The block bootstrap, applied to Wolff output where the small $\tau_{\text{int}}$ permits short blocks and many independent samples, provides honest standard errors that successfully recover the exact critical temperature within statistical uncertainty.

### 13.3 Methodological choices

Following the course instructor's directive to prioritize implementation depth over breadth, this project focuses on the Metropolis vs Wolff comparison as the primary scientific deliverable, with block bootstrap as the single advanced UQ component. Simulated annealing and histogram reweighting were demoted from the original proposal. The Rcpp implementations of both samplers achieve approximately 100-300x speedup over pure-R reference implementations, making the systematic studies across $(L, T)$ computationally feasible. All results use seed-controlled RNG (R's RNG threaded through Rcpp via `GetRNGstate`/`PutRNGstate`) for full reproducibility.

## 14 References


Kunsch, H. R. (1989). The jackknife and the bootstrap for general stationary observations. *Annals of Statistics*, 17(3), 1217-1241.

Newman, M. E. J., & Barkema, G. T. (1999). *Monte Carlo Methods in Statistical Physics*. Oxford University Press.

Nightingale, M. P., & Bl"ote, H. W. J. (1996). Dynamic exponent of the two-dimensional Ising model and Monte Carlo computation of the subdominant eigenvalue of the stochastic matrix. *Physical Review Letters*, 76(24), 4548-4551.

Onsager, L. (1944). Crystal statistics. I. A two-dimensional model with an order-disorder transition. *Physical Review*, 65(3-4), 117-149.

Sokal, A. D. (1997). Monte Carlo methods in statistical mechanics: foundations and new algorithms. In *Functional Integration: Basics and Applications* (pp. 131-192). Plenum Press.

Wolff, U. (1989). Collective Monte Carlo updating for spin systems. *Physical Review Letters*, 62(4), 361-364.

Yang, C. N. (1952). The spontaneous magnetization of a two-dimensional Ising model. *Physical Review*, 85(5), 808-816.